



UNIVERSIDADE DA CORUÑA

Facultade de Informática

Máster Universitario en Enxeñaría Informática

TRABALLO FIN DE MÁSTER

Personalización, explicabilidade y sostenibilidad en sistemas de recomendación

Estudiante: David Esteban Martínez
Dirección: Amparo Alonso Betanzos
Bertha Guijarro Berdiñas

A Coruña, febrero de 2025.

A mi novio, Elio

Agradecimientos

Este trabajo será probablemente el fin de mi etapa como estudiante, quiero agradecer a todos los que me han acompañado en este camino, mi hermano, que decidió copiarme y hacer el mismo grado y máster; Jan, Suso, Román, Jorge, Samuel, y Gayoso, por acompañarme desde el inicio de mi recorrido como informático; a mi tío David, por no solo instarme a que empezase un Grado, si no por llevarme incontables días a la universidad en su coche, primero por estudios y luego por trabajo; a Alex, el señor Dopico, Rubén, Adrián, Samuel, Samuel-Ismael, Abel, y Jorge, compañeros y amigos en el trabajo que me han escuchado y ayudado durante los largos meses que duró este trabajo; a Amparo y Bertha, mis directoras que no perdieron la paciencia a pesar de los lentos progresos; a mi novio Elio, que ya me acompañó durante mi Trabajo de Fin de Grado y me ha acompañado también durante este, esta vez librándose de hacer encuestas; y finalmente a la Facultad de Informática, la llevaré siempre en mi corazón.

Resumen

Este trabajo aborda la problemática de la transparencia en los [Sistemas de Recomendación \(SR\)](#), proponiendo técnicas innovadoras para mejorar la explicabilidad visual, con un enfoque en la sostenibilidad. Para ello, hemos llevado a cabo un estudio con tres modelos del Estado del Arte ([ELVis](#), [MF-ELVis](#) y [BRIE](#)) aplicados a reseñas de Tripadvisor. Las técnicas empleadas se centran en optimizar la calidad de los datos de entrada, a través de tres estrategias principales: 1) Selección de nuevos positivos y negativos con Aprendizaje Positivo y Sin Etiquetas, 2) Aumentación de datos por transformación de imágenes o IA Generativa, y 3) Mejora de *embeddings* de imágenes. Los resultados demuestran que la mejora de *embeddings* incrementa el rendimiento en un 30%, reduce el tiempo de entrenamiento, y disminuye las emisiones y el consumo en un 20% y 15% en entrenamiento e inferencia, respectivamente. La combinación con las demás técnicas genera una mejora adicional del 5% en el rendimiento y reduce significativamente las épocas de entrenamiento, disminuyendo las emisiones hasta un 60% adicional. Finalmente, estas técnicas han sido incluidas en una librería pública de carácter modular, permitiendo su utilización en entornos de producción, así como en investigaciones incrementales que partan de este trabajo.

Abstract

This work addresses the issue of transparency in Recommender Systems (RS), proposing innovative techniques to improve visual explainability, with a focus on sustainability. To this end, we conducted a study with three state-of-the-art models ([ELVis](#), [MF-ELVis](#), and [BRIE](#)) applied to Tripadvisor reviews. The employed techniques focus on optimizing the quality of input data through three main strategies: 1) Selection of new positives and negatives using Positive and Unlabeled Learning, 2) Data augmentation through image transformation or Generative AI, and 3) Enhancement of image *embeddings*. The results show that improving *embeddings* increases performance by 30%, reduces training time, and decreases emissions and consumption by 20% and 15% in training and inference, respectively. The combination with the other techniques provides an additional 5% performance improvement and significantly reduces training epochs, lowering emissions by up to an additional 60%. Finally, these techniques have been included in a publicly available modular library, allowing their use in production environments as well as in incremental research building upon this work.

Palabras clave:

- Aprendizaje Automático
- Sostenibilidad
- Inteligencia Artificial Explicable
- Inteligencia Artificial Verde
- Inteligencia Artificial Frugal
- Aprendizaje Positivo y Sin Etiquetas
- Aumentación de los Datos
- Sistemas de Recomendación
- Inteligencia Artificial Generativa

Keywords:

- Machine Learning
- Sustainability
- Explainable Artificial Intelligence
- Green Artificial Intelligence
- Frugal Artificial Intelligence
- Positive Unlabeled Learning
- Data Augmentation
- Recommender Systems
- Generative Artificial Intelligence

Índice general

1	Introducción	1
2	Estudio de Viabilidad, Metodología y herramientas	3
2.1	Viabilidad Técnica	3
2.1.1	Herramientas y Tecnologías disponibles	3
2.1.2	Viabilidad Económica	4
2.1.3	Viabilidad Operativa	5
2.2	Metodología	6
2.2.1	Cross Industry Standard Process for Data Mining (CRISP-DM)	6
2.2.2	Aplicación de CRISP-DM al Proyecto	8
2.3	Herramientas utilizadas	9
2.3.1	Control de Versiones: Github	10
2.3.2	Lenguaje: Python	10
2.3.3	Entorno de desarrollo: PyCharm	10
2.3.4	Pytorch [1]	10
2.3.5	Pytorch Lightning [2]	10
2.3.6	Pickle [3]	10
2.3.7	Pandas [4]	11
2.3.8	Numpy [5]	11
2.3.9	CodeCarbon [6]	11
2.3.10	HuggingFace [7]	11
3	Background y Fundamentos Teóricos	13
3.1	Sistemas de Recomendación	13
3.1.1	Tipos de Sistemas de Recomendación	13
3.1.2	Algoritmos Clave en Sistemas de Recomendación	15
3.2	Explicabilidad Visual en Sistemas de Recomendación	15
3.2.1	Definición y Clasificación de la Explicabilidad en SR	16

3.2.2	Ranking de imágenes en explicabilidad visual	16
3.2.3	Explaining Likings Visually (ELVis) [8]	18
3.2.4	Matrix Factorization ELVis (MF-ELVis) [9]	18
3.2.5	Bayesian Ranking of Images for Explainability (BRIE) [10]	20
3.3	Modelos de <i>Embedding</i> de Imágenes	22
3.4	Técnicas de mejora de calidad de los Datos	22
3.4.1	Aprendizaje Positivo y Sin Etiquetas	23
4	Técnicas propuestas	27
4.1	Aprendizaje Positivo y Sin Etiquetas	27
4.1.1	Selección de negativos mediante similitud de imágenes	28
4.1.2	Selección de nuevos positivos mediante similitud de usuarios	29
4.2	Aumentación de los Datos	31
4.2.1	Aumentación de los Datos por transformación de imágenes	31
4.2.2	Inteligencia Artificial Generativa	32
4.3	Mejora en los <i>embedding</i> de imágenes	37
5	Configuración experimental	39
5.1	Datos	39
5.2	Métricas	40
5.2.1	Área bajo la Curva de Característica Operativa del Receptor (AUC-ROC)	41
5.2.2	Sensibilidad	42
5.2.3	Normalized Discounted Cumulative Gain (NDCG)	42
5.2.4	Bilingual Evaluation Understudy (BLEU) y character n-gram F-score (CHRF)	43
5.2.5	Métricas de sostenibilidad	44
5.3	Híper-parámetros	45
5.3.1	Métrica de similitud entre imágenes	45
6	Resultados	49
6.1	Rendimiento global de los modelos	49
6.2	Impacto en el consumo energético y emisiones	50
6.3	Consumo energético de las técnicas propuestas	51
6.4	Consideraciones finales sobre el costo de las técnicas	51
7	Entregable: Librería para mejora de sistemas de explicabilidad visual	57
7.1	Buenas Prácticas en el Desarrollo de la Librería	58
7.1.1	Cumplimiento de Estándares de Código	58
7.1.2	Modularidad y Reutilización del Código	59

7.2	Diseño y Arquitectura de la Librería	59
7.2.1	Estructura General	59
7.2.2	Módulo de aumentación de Datos	60
7.2.3	Módulo de Generación de Embeddings	60
7.2.4	Módulo de Selección de Negativos con PU Learning	60
7.2.5	Módulo de Selección de Positivos con PU Learning	61
7.3	Guía de Uso	61
7.3.1	Módulo de Aumentación de Datos	61
7.3.2	Módulo de Generación de <i>Embeddings</i> (<code>new_embeddings.py</code>) . .	63
7.3.3	Módulo de Selección de Negativos con PULearning (<code>pu_negatives.py</code>)	63
7.3.4	Módulo de Selección de Positivos con PULearning (<code>pu_positives.py</code>)	64
8	Conclusiones	67
8.1	Contribuciones principales	67
8.2	Conclusiones del alumno	69
8.3	Trabajo Futuro	69
	Lista de acrónimos	71
	Bibliografía	73

Índice de figuras

3.1	Arquitectura y topología del modelo ELVis. Obtenida de [10].	19
3.2	Arquitectura y topología del modelo MF-ELVis. Obtenida de [10].	20
3.3	Arquitectura y topología del modelo BRIE. Obtenida de [10].	21
4.1	Enfoque de PU Learning personalizado para seleccionar de forma fiable imágenes negativas (malas explicaciones) para cada usuario en el contexto de la explicación de la recomendación. En este ejemplo, se muestra la frontera de decisión personalizada para la selección de ejemplos negativos fiables para dos usuarios A y B con diferentes preferencias de explicación. Obtenida del artículo original de la técnica para ELVis [11].	30
4.2	Transformaciones utilizadas sobre las imágenes de los conjuntos de datos como aumento de los datos.	32
4.3	Imagen obtenida con la consulta 4.2.2 con StableDiffusion3.5.	33
4.4	Imagen obtenida con la consulta 4.2.2 con StableDiffusion1.5.	34
4.5	Imagen obtenida con la consulta 4.2.2 con StableDiffusion1.5 al seleccionar un tamaño de 150x150.	35
4.6	Imagen obtenida con la consulta 4.2.2 con aMUSEd256.	36
4.7	Imágenes obtenidas con el modelo aMUSEd256 para diferentes consultas creadas a partir de reseñas de usuarios.	36
4.8	Arquitectura del modelo InceptionResnetV2, obtenida de [12].	37
4.9	Arquitectura del modelo ViT, obtenida de [13].	38

Índice de tablas

2.1	Costes estimados del proyecto basados en el número de horas de trabajo. . . .	5
5.1	Resumen de los datasets utilizados en términos de usuarios, restaurantes, fotografías y promedio de fotografías por usuario.	41
5.2	Partición de los datasets en entrenamiento, validación y prueba.	41
5.3	Hiper-parámetros de los modelos ELVis, MF-ELVis, y BRIE, junto con las técnicas aplicadas y la reducción del hiper-parámetro de número de épocas correspondiente a cada técnica.	46
6.1	Resultados obtenidos con el conjunto de datos de Gijón.	52
6.2	Resultados obtenidos con el conjunto de datos de Barcelona.	53
6.3	Resultados obtenidos con el conjunto de datos de Madrid.	54
6.4	Resultados de tiempo, emisiones, y consumo, para la etapa de entrenamiento del conjunto de datos de Barcelona en todos los modelos de explicabilidad visual utilizados.	54
6.5	Resultados de tiempo, emisiones, y consumo, para 10 ejecuciones de la inferencia del conjunto de prueba, para el conjunto de datos de Barcelona en todos los modelos de explicabilidad visual utilizados.	55
6.6	Resultados de tiempo, emisiones, y consumo, para las técnicas presentadas utilizando los <i>embeddings</i> producidos por el modelo ViT Large 14 para el conjunto de datos de Barcelona, Aumentación por genAI incluye la generación de imágenes y su posterior transformación a <i>embeddings</i> y adición al conjunto de datos.	55
6.7	Resultados de tiempo de entrenamiento ,consumo para las mejores aproximaciones tras la optimización del hiper-parámetro de épocas de entrenamiento, para el conjunto de datos de Gijón y con los <i>embeddings</i> producidos con ViT Large 14.	55

6.8	Resultados de tiempo de entrenamiento ,consumo para las mejores aproximaciones tras la optimización del hiper-parámetro de épocas de entrenamiento, para el conjunto de datos de Barcelona y con los <i>embeddings</i> producidos con ViT Large 14.	56
6.9	Resultados de tiempo de entrenamiento ,consumo para las mejores aproximaciones tras la optimización del hiper-parámetro de épocas de entrenamiento, para el conjunto de datos de Madrid y con los <i>embeddings</i> producidos con ViT Large 14.	56

Introducción

EN la última década, los sistemas de recomendación se han convertido en una parte fundamental de la interacción digital, personalizando experiencias en plataformas de comercio electrónico, servicios de streaming, redes sociales y más. Estos sistemas procesan grandes volúmenes de datos para proporcionar sugerencias relevantes a los usuarios, mejorando la satisfacción de los mismos, incrementando el tiempo de permanencia en la plataforma y optimizando las tasas de conversión [14]. Sin embargo, pese a su utilidad, muchos sistemas de recomendación enfrentan una barrera significativa: la falta de explicaciones claras sobre las decisiones tomadas para realizar dichas recomendaciones. Esta carencia de explicabilidad puede generar desconfianza y reducir la predisposición de los usuarios a aceptar las sugerencias [15]. La explicabilidad en los sistemas de recomendación se refiere a la capacidad de un modelo para justificar o razonar las recomendaciones ofrecidas a los usuarios, proporcionando información comprensible sobre el proceso subyacente. Esta característica no solo aumenta la confianza del usuario, sino que también mejora la transparencia del sistema y facilita la detección de posibles sesgos en el modelo, con otras funciones como crear en el usuario una sensación de confianza, aumentar la lealtad de los usuarios, agilizar y facilitar los procesos de encontrar el producto deseado, e incluso convencer a los usuarios a la prueba o compra de un producto recomendado [16]. Esta explicabilidad puede ser una parte intrínseca del modelo, siendo este mismo un modelo explicable, como los árboles de decisión, modelos de regresión lineal, o modelos más complejos como las Máquinas de Incrementación Explicable (Explainable Boosting Machine o EBM por sus siglas en inglés) [17]; o más comúnmente en modelos de Aprendizaje Automático (AA), esta explicabilidad es llevada por un modelo externo de explicabilidad, como son los conocidos modelos SHapley Additive exPlanations (SHAP) [18] y Local Interpretable Model-agnostic Explanation (LIME) [19]. Estos modelos de explicabilidad son capaces de explicar cualquier modelo de AA, y los sistemas de recomendación no son una excepción [20, 21, 22, 23]. Un problema común de estas técnicas es la naturaleza de las explicaciones que proveen, basadas en la importancia de los diferentes atributos de los datos

a explicar, algo que dificulta el entendimiento de estas explicaciones a los propios usuarios que busca ayudar al no ser fácilmente interpretable [20]. En este contexto, la explicabilidad visual surge como una estrategia innovadora que utiliza elementos visuales, como imágenes relevantes para el usuario, para justificar las recomendaciones. Este enfoque tiene el potencial de hacer las explicaciones más intuitivas y accesibles, especialmente en dominios donde los datos visuales tienen un papel prominente, como la fotografía, los viajes o la moda.

Aunque la mejora de sistemas de [AA](#) es un tema muy explorado en la literatura, no ocurre lo mismo con los sistemas de explicabilidad, ya que la tendencia actual se centra en crear nuevos modelos con mejores capacidades explicativas, en lugar de desarrollar técnicas para mejorar el rendimiento de los modelos existentes. Una de las soluciones más intuitivas, y común en el ámbito del [AA](#), es el desarrollo de modelos más grandes o la incorporación de más datos. Sin embargo, este enfoque no es sostenible, especialmente en un contexto global donde la eficiencia energética y la sostenibilidad son prioridades clave. Las operaciones intensivas en datos y el aumento en la capacidad de los modelos no solo incrementan el consumo de recursos computacionales, sino que también amplían la huella de carbono asociada al entrenamiento y despliegue de los sistemas [24]. La sostenibilidad, tanto ambiental como operativa, es fundamental en el diseño de cualquier tecnología moderna. Desde una perspectiva ambiental, se deben desarrollar soluciones que minimicen el impacto ecológico de los sistemas de aprendizaje automático. Por otro lado, desde una óptica operativa, la sostenibilidad también implica garantizar que las soluciones sean replicables y escalables, incluso en entornos con recursos limitados.

La motivación principal de este proyecto radica en la necesidad de abordar las limitaciones actuales de los sistemas de recomendación en términos de explicabilidad y sostenibilidad. En cuanto a la explicación visual, su potencial para mejorar la confianza del usuario y la aceptación de recomendaciones es indiscutible, aunque aún es un campo joven y con mucho camino por explorar, por ello es importante tomar un rumbo que busque una mejoría sostenible, en vez del uso únicamente de modelos más grandes o conjuntos de datos más extensos para mejorar la calidad de las explicaciones, algo que puede ser insostenible desde una perspectiva energética. Este trabajo busca contribuir al Estado del Arte mediante el desarrollo de técnicas innovadoras que optimicen el rendimiento de los sistemas de explicabilidad visual, priorizando su sostenibilidad, siendo el objetivo la creación de soluciones escalables y eficientes que puedan ser adoptadas tanto en entornos empresariales como en investigaciones futuras.

La relevancia de este trabajo, se ve respaldada por la creciente importancia de la sostenibilidad en la computación. La minimización del impacto ambiental de los sistemas de aprendizaje automático es un tema prioritario para la comunidad científica, así como el de la transparencia de los diferentes sistemas de [Inteligencia Artificial \(IA\)](#), con la creación del “AI Act” en la Unión Europea [25].

Estudio de Viabilidad, Metodología y herramientas

ANTES de emprender cualquier proyecto de investigación o desarrollo tecnológico, resulta fundamental llevar a cabo un estudio de viabilidad exhaustivo. Este análisis permite evaluar las posibilidades de éxito del proyecto en términos técnicos, económicos y operativos, así como anticipar posibles desafíos que puedan surgir durante su ejecución [26]. Para ello, llevamos a cabo una serie de evaluaciones de la viabilidad del proyecto que garanticen que los objetivos planteados sean alcanzables dentro de las limitaciones y recursos disponibles. A continuación, expondremos el análisis desde tres perspectivas principales: viabilidad técnica, económica, y operativa.

2.1 Viabilidad Técnica

La viabilidad técnica del proyecto depende de las herramientas disponibles y los datos necesarios. Las herramientas utilizadas se explicarán más a fondo en la Sección 2.3, y los datos en el Capítulo 3.

2.1.1 Herramientas y Tecnologías disponibles

- **Frameworks de Aprendizaje Automático:** Empleamos Pytorch-Lightning como herramienta principal para el desarrollo de modelos y experimentos. Este framework, ampliamente reconocido, permite una implementación modular y eficiente de modelos complejos, facilitando su escalabilidad y depuración.
- **Hardware:** Todos los experimentos del proyecto fueron ejecutados en un ordenador personal con capacidades de procesamiento estándar, destacando la presencia de una tarjeta gráfica NVIDIA 3050 con cores CUDA y 4GB de VRAM.

- **Datasets:** El proyecto sería aplicable en cualquier escenario en el que existan imágenes procedentes de los usuarios del sistema de recomendación. En este caso, se utilizaron datos públicos de Tripadvisor, previamente validados en investigaciones académicas en el mismo contexto que este proyecto, la explicabilidad visual en sistemas de recomendación.
- **Librerías Complementarias:** Adicionalmente, se utilizaron librerías como Torch [1], para optimizar procesos de cálculo matricial y acelerar la implementación de los métodos desarrollados, o Pickle [3], para la serialización y de-serialización de objetos de Python, útil para guardar modelos en sus estados finales de entrenamiento, además de la compresión que ofrece a los datasets utilizados.

2.1.2 Viabilidad Económica

La viabilidad económica del proyecto se fundamenta en la minimización de costos y en el uso de recursos accesibles, sin comprometer la calidad de los resultados.

Costes Asociados

- **Hardware:** No se requirió la adquisición de equipamiento costoso. Los recursos existentes, como un ordenador personal, son suficientes para todo el desarrollo.
- **Software:** Al utilizarse exclusivamente software de código abierto, no hay costes adicionales asociados a licencias o suscripciones.
- **Datos:** Los datasets de Tripadvisor son públicos y han sido utilizados previamente en investigación, evitando así gastos por adquisición de datos.
- **Trabajo:** El trabajo fue realizado por el estudiante a tiempo completo en el intervalo temporal entre el 1 de Octubre y el 14 de Febrero, con 40 horas semanales, por lo que siendo 20 semanas esto equivale a 800 horas de trabajo. Hubo desde el inicio del trabajo una reunión semanal con dos tutoras de duración aproximada de 30 minutos cada una, lo que equivale a unas 20 horas de trabajo, ya contadas en el tiempo de trabajo del estudiante. Además de esto, la revisión de este documento llevó a 8 horas de trabajo adicional por parte de las tutoras. Con ello, podemos hacer una estimación de los costes totales del proyecto en la Tabla 2.1

Replicabilidad y Escalabilidad

El proyecto ha sido diseñado para ser replicable en entornos con recursos limitados. La ausencia de dependencia de servicios de nube o infraestructura costosa garantiza que las téc-

Trabajador	Horas x Salario	salario Final
Estudiante	800 x 22€	17.600€
Tutoras	28 x 40€	1.148€
Total		18.748€

Tabla 2.1: Costes estimados del proyecto basados en el número de horas de trabajo.

nicas desarrolladas puedan ser adoptadas por una amplia gama de usuarios y organizaciones, incluyendo startups o instituciones académicas con presupuestos restringidos.

ROI (Retorno de Inversión)

El desarrollo de una librería profesional presenta un gran potencial para generar beneficios tanto a corto como a largo plazo. En el ámbito académico, puede impulsar la producción de publicaciones en conferencias y revistas especializadas, fortaleciendo el impacto científico. En el ámbito empresarial, la integración de la librería en sistemas comerciales ofrecería mejoras significativas en la experiencia del usuario, incrementando la confianza en las recomendaciones. Esto podría traducirse en mayores tasas de conversión, fidelización de clientes y un valor añadido para las organizaciones que implementen la solución.

2.1.3 Viabilidad Operativa

La viabilidad operativa del proyecto se evalúa considerando la capacidad de ejecutar el trabajo dentro de los plazos y requisitos establecidos, así como la facilidad de integración de los resultados en futuros desarrollos. El enfoque del proyecto se centra en el desarrollo de una librería profesional, lo que reduce la complejidad operativa y permite un avance más controlado al eliminar la necesidad de desplegar un sistema prototipo. La organización del trabajo en fases bien definidas asegura el cumplimiento de los objetivos dentro del tiempo asignado. Además, el acceso a los recursos del CITIC proporciona soporte técnico crucial para abordar limitaciones potenciales, y la comunidad de PyTorch-Lightning facilita la resolución de problemas técnicos y la incorporación de mejores prácticas. Se dio prioridad a la elaboración de una documentación estructurada y detallada, con el objetivo de garantizar que las técnicas desarrolladas sean fácilmente comprensibles e integrables, tanto para su aplicación en trabajos futuros como para su utilización por parte de otros desarrolladores. Finalmente, se establecieron planes de mitigación de riesgos específicos para evitar contratiempos indeseados que afectasen a la entrega del proyecto en la fecha deseada.

2.2 Metodología

Este trabajo fue desarrollado bajo el marco metodológico [Cross Industry Standard Process for Data Mining \(CRISP-DM\)](#) [27], ampliamente utilizado en proyectos de ciencia de datos y aprendizaje automático debido a su enfoque sistemático y flexible. Antes de hablar sobre como aplicamos esta metodología a nuestro trabajo, explicaremos sus diferentes fases, interacciones entre fases, y su aplicabilidad general

2.2.1 Cross Industry Standard Process for Data Mining (CRISP-DM)

CRISP-DM es un proceso cíclico compuesto por seis fases principales, diseñadas para guiar desde la comprensión del problema hasta la implementación de una solución robusta y evaluada. Estas fases no son necesariamente lineales; su interacción permite iteraciones para optimizar resultados.

Comprensión del Negocio

Primera fase de la metodología, el objetivo en esta fase es entender el contexto y requerimientos del problema a resolver, así como los objetivos del cliente o usuario final, estableciendo criterios de éxito y restricciones operativas. Las tareas principales son:

- Definición del problema
- Identificación de las partes interesadas
- Análisis de los objetivos del negocio y transformación en objetivos técnicos

Comprensión de los Datos

Fase centrada en explorar los datos disponibles para determinar su calidad, relevancia y limitaciones. Tanto esta fase como la anterior, Comprensión de Negocio, son de extrema importancia, ya que dedicar esfuerzos adecuados a estas fases puede reducir al mínimo los gastos indirectos relacionados. Las tareas principales son:

- Revisión de la estructura de los datos
- Identificación de sesgos y patrones iniciales
- Evaluación de la suficiencia de los datos para abordar el problema planteado

Preparación de los Datos

En esta etapa, se lleva a cabo un proceso selección, limpieza y transformación de los datos para crear un conjunto adecuado que pueda ser utilizado por los algoritmos de modelado. Según estimaciones, esta fase suele costar entre el 50-70% del tiempo y esfuerzo de un proyecto [28]. Las tareas principales son:

- Integración y consolidación de diversas fuentes de datos.
- Normalización y codificación de variables categóricas.
- Generación de nuevas características relevantes para el modelado.

Modelado

Esta fase implica el desarrollo de modelos predictivos o descriptivos. La selección del algoritmo depende de los objetivos definidos y las características de los datos. Las tareas principales son:

- Selección de algoritmos
- Ajuste de hiperparámetros
- Obtención de resultados

Evaluación

Tras obtener resultados de los modelos en la fase anterior, se analiza su calidad según métricas definidas previamente, asegurando el cumplimiento de los requisitos de negocio y técnicos. Las tareas principales son:

- Evaluación del rendimiento del modelo
- Comparación con métodos existentes

Despliegue

Fase final de la metodología, la solución se implementa en un entorno de producción o se documenta para uso posterior.

Interacciones entre fases

La naturaleza iterativa de la metodología [CRISP-DM](#) permite en vez de avanzar linealmente como una metodología tradicional de tipo cascada, retroceder a pasos anteriores del proceso utilizando los nuevos conocimientos para obtener los mejores resultados posibles previos al despliegue. Según las guías de la metodología esto se puede hacer en 3 momentos concretos:

- Tras la fase de Comprensión de los Datos, volver a la fase de Comprensión del Negocio.
- Tras la fase de Modelado, volver a la fase de preparación de los Datos.
- Tras la fase de Evaluación, volver al principio, a la fase de Comprensión del Negocio

2.2.2 Aplicación de [CRISP-DM](#) al Proyecto

Comprensión del Negocio

En el contexto de este proyecto, se identificó que la falta de explicaciones claras en los sistemas de recomendación disminuye la confianza del usuario [15]. Además, se definió como problema principal el balance entre el rendimiento y la sostenibilidad de las técnicas de explicabilidad visual. De esta fase, derivamos los siguientes objetivos:

- Mejorar la calidad de las explicaciones visuales.
- Mantener o reducir el costo computacional

Comprensión de los Datos

Los datos utilizados provienen de conjuntos de datos públicos de Tripadvisor, obtenidos de [8]. Estos datos se centran en las reseñas de usuarios y las imágenes asociadas. En nuestra revisión inicial identificamos un desbalance significativo en la distribución de imágenes por usuario, ya que el 50% de imágenes provienen de usuarios con menos de 5 imágenes entre sus reseñas, siendo este grupo de usuarios un 90% del total de usuarios. Además de esto, el mero hecho de obtener datos de terceros implicó un riguroso procedimiento para entender las transformaciones previas y agrupamientos realizados.

Preparación de los Datos

La fase más importante en nuestro proyecto, en la que invertimos la mayor parte del tiempo de trabajo. Esto es así ya que gran parte de las soluciones propuestas dependen de mejorar la calidad de los datos. Estas soluciones propuestas son:

- Selección de nuevos negativos mediante [PU Learning](#)

- Selección de nuevos positivos mediante [PU Learning](#)
- Aumentación de los Datos por transformación de imágenes
- Aumentación de los Datos por Inteligencia Artificial Generativa
- Mejora en los *embedding* de imágenes

Las soluciones están expuestas en detalle en el [Capítulo 4](#)

Modelado

Durante esta fase llevamos a cabo el entrenamiento de los modelos con los datos obtenidos en la fase anterior. Los modelos utilizados fueron: ELVis [8], MF-ELVis [9], y BRIE [10]. Estos modelos están explicados en detalle en el [Capítulo 3](#)

Evaluación

Nuestra evaluación de resultados se basó en las siguientes métricas:

- Recall@10: Proporción de ítems relevantes recuperados entre los 10 primeros.
- NDCG@10: Medida de la calidad de las recomendaciones, ponderada por la posición de los ítems relevantes entre los 10 primeros.
- AUC: Área bajo la curva ROC, mide la capacidad de clasificación de un modelo.
- BLEU: Métrica para evaluar la similitud entre una secuencia generada y una secuencia de referencia.
- CHRF: Métrica basada en la coincidencia de n-gramas de caracteres entre textos.

Todas estas métricas están explicadas en el [Capítulo 6](#)

Despliegue

En vez del despliegue de la solución en un entorno de producción, en esta fase lo que se llevo a cabo fue la elaboración y estructuración de la librería que aporta soluciones al problema presentado durante la fase de Comprensión del Negocio

2.3 Herramientas utilizadas

Gran parte de la planificación de un proyecto recae en la elección de herramientas a utilizar, estas son las más destacables en este trabajo.

2.3.1 Control de Versiones: Github

Plataforma de desarrollo colaborativo basada en la nube que permite gestionar proyectos de software y controlar versiones de código. Durante este trabajo, se utilizó GitHub para almacenar, compartir y realizar un seguimiento de las versiones del código fuente.

2.3.2 Lenguaje: Python

Lenguaje de programación de alto nivel, reconocido por su sintaxis simple y legible, lo que facilita el desarrollo rápido de aplicaciones complejas. Único lenguaje de desarrollo en este proyecto, usado por la presencia de implementaciones de los modelos utilizados en este lenguaje, además de por su extenso ecosistema de bibliotecas y herramientas especialmente útiles en las tareas realizadas.

2.3.3 Entorno de desarrollo: PyCharm

[Entorno de Desarrollo Integrado \(IDE\)](#) específico para Python, propiedad de JetBrains, proporciona herramientas avanzadas para la programación, como autocompletado de código, depuración y análisis de errores. Su elección frente a otros [IDE](#) se debe a la familiaridad del estudiante con la herramienta, además de la posibilidad de utilizar una licencia gratuita por la propia condición de ser estudiante.

2.3.4 Pytorch [1]

Biblioteca de Aprendizaje Profundo que permite la creación y entrenamiento de modelos de redes neuronales mediante un enfoque dinámico y flexible. Necesaria para utilizar los modelos de Explicabilidad Visual [ELVis](#), [MF-ELVis](#), y [BRIE](#).

2.3.5 Pytorch Lightning [2]

Extensión de Pytorch que simplifica el desarrollo de modelos de Aprendizaje Profundo, separando la lógica del modelo y la gestión del entrenamiento. De nuevo, necesaria para utilizar los modelos de Explicabilidad Visual.

2.3.6 Pickle [3]

Módulo de Python utilizado para la serialización y deserialización de objetos. Fue necesario para guardar los modelos tras terminar el entrenamiento y poder hacer la fase de Test por separado, además de para almacenar en espacio reducido los conjuntos de datos como objetos Dataframe de Pandas, y en formato de arrays de Numpy las imágenes transformadas en vectores de características latentes.

2.3.7 Pandas [4]

Biblioteca de Python para la manipulación y análisis de datos estructurados. Necesaria para trabajar con los conjuntos de datos originales además de con las modificaciones presentadas para aumentar el rendimiento de los modelos.

2.3.8 Numpy [5]

Biblioteca fundamental para el cálculo numérico en Python, que proporciona soporte para operaciones con arrays multidimensionales y funciones matemáticas avanzadas. Necesaria para trabajar con los arrays de imágenes vectorizadas.

2.3.9 CodeCarbon [6]

Herramienta diseñada para medir la huella de carbono generada por programas de software, especialmente durante el entrenamiento de modelos de [Aprendizaje Automático](#). Empleamos esta herramienta para monitorizar el impacto energético y las emisiones de CO2 de nuestras soluciones, de forma análoga a los trabajos que presentan el Estado del Arte en el campo [9, 10].

2.3.10 HuggingFace [7]

Plataforma y biblioteca líder en el desarrollo de modelos de lenguaje y aprendizaje profundo. Proporciona herramientas y modelos preentrenados, especialmente útiles para tareas de [Procesamiento de Lenguaje Natural](#) (NLP) e Inteligencia Artificial Generativa. En este trabajo, se emplearon sus capacidades para gestionar y adaptar modelos preentrenados, tanto de embedding de imágenes como de generación de imágenes.

Background y Fundamentos Teóricos

ESTE capítulo está dedicado a introducir los conceptos y tecnologías claves que sustentan el trabajo.

3.1 Sistemas de Recomendación

Los **Sistemas de Recomendación (SR)** son una subdisciplina de la **IA** y el **AA** que tiene como objetivo principal predecir las preferencias o necesidades de los usuarios y proporcionar sugerencias personalizadas. Estos sistemas han ganado una relevancia significativa en una amplia variedad de dominios, como el comercio electrónico, plataformas de streaming y servicios de educación en línea, debido a su capacidad para manejar grandes volúmenes de datos y personalizar experiencias [29].

3.1.1 Tipos de **Sistemas de Recomendación**

Hay diferentes tipos de **SR**, dependiendo del mecanismo utilizado para computar recomendaciones.

Recomendación Basada en Contenido

La recomendación basada en contenido o **Content Based (CB)** utiliza información sobre los objetos o ítems que el usuario ha consumido previamente para sugerir ítems similares. Estos sistemas dependen de representaciones de características que describen los ítems y los intereses del usuario. Por ejemplo, en una aplicación de libros, si un usuario lee novelas de ciencia ficción, se le recomendarán otros libros del mismo género [30]. Una característica importante de este tipo de **SR** es que trabajan con información explícita, características reales del

entorno, como la demografía o edad del usuario, lo que hace que esta información sea interpretable. Formalmente, un [SR](#) basado en contenido puede representarse mediante un conjunto de vectores de características $P = p_1, p_2, \dots, p_n$ que describen cada ítem, y un perfil de usuario $U = u_1, u_2, \dots, u_n$ que refleja las preferencias del usuario en términos de estas características. La similitud entre ítems y el perfil del usuario generalmente se mide mediante métricas como el coseno de similitud o la distancia euclídea.

Ventajas

- Alto grado de personalización.
- Independencia de las opiniones de otros usuarios.

Desventajas

- Problemas de sobreajuste.
- Dificultad para recomendar ítems que no se ajusten a patrones preexistentes.

Filtrado Colaborativo

El filtrado colaborativo o [Collaborative Filtering \(CF\)](#) se basa en la idea de que los usuarios que han mostrado intereses similares en el pasado probablemente compartan intereses futuros [31]. Este enfoque puede dividirse en dos métodos principales:

1. **Basado en usuarios:** Busca usuarios con preferencias similares al usuario objetivo y recomienda ítems que estos han consumido.
2. **Basado en ítems:** Encuentra ítems similares a aquellos que el usuario objetivo ha evaluado positivamente.

Técnicamente, el [CF](#) utiliza una matriz de interacción usuario-ítem R , donde r_{ij} representa la evaluación o interacción del usuario i con el ítem j . Las técnicas comunes incluyen la descomposición matricial, como la factorización en valores singulares (SVD), para predecir valores desconocidos en R [32].

Ventajas

- No requiere representación detallada de los ítems.
- Capacidad para descubrir patrones latentes en las preferencias de los usuarios.

Desventajas

- Problema del “comienzo en frío” (cold start) para nuevos usuarios o ítems.
- Escalabilidad limitada en sistemas con grandes volúmenes de datos.

Enfoques Híbridos

Los sistemas híbridos combinan múltiples métodos de recomendación para mitigar las limitaciones inherentes de cada uno [33]. Por ejemplo, un sistema puede integrar el CB y el CF para equilibrar la personalización y la diversidad de las recomendaciones.

3.1.2 Algoritmos Clave en Sistemas de Recomendación

Factorización Matricial

La factorización matricial es una de las técnicas más utilizadas en el filtrado colaborativo. Consiste en descomponer la matriz de interacciones $R \in \mathbb{R}^{m \times n}$ en dos matrices latentes $S \in \mathbb{R}^{m \times k}$ y $Q \in \mathbb{R}^{n \times k}$, donde k es la dimensionalidad latente. El producto de estas matrices, $R' = S \cdot Q^T$, aproxima las interacciones originales [32]. Los valores desconocidos en R pueden predecirse mediante:

$$\hat{r}_{up} = s_u^T q_p$$

donde s_u y q_p representan los vectores latentes del usuario u y el ítem p , respectivamente.

Redes Neuronales

Con el auge del Aprendizaje Profundo (Deep Learning), las redes neuronales, modelos de inteligencia artificial que buscan replicar el funcionamiento del cerebro humano, han sido ampliamente adoptadas para sistemas de recomendación. Estas redes pueden modelar relaciones complejas y no lineales entre usuarios e ítems. Los ejemplos más prominentes son:

- Redes Neuronales Recurrentes o [Recurrent Neural Networks \(RNN\)](#): Eficaces para recomendaciones secuenciales o basadas en el historial temporal
- Redes Neuronales Convolucionales o [Convolutional Neural Networks \(CNN\)](#): Adecuadas para capturar patrones especiales en datos estructurados o de contenido [34]

Modelos Basados en Grafos

Los grafos permiten modelar relaciones complejas entre usuarios y ítems en forma de un grafo bipartito. Las técnicas de incrustación, como DeepWalk [35] o Node2Vec [36], se utilizan para representar nodos en un espacio latente que facilita las recomendaciones.

3.2 Explicabilidad Visual en Sistemas de Recomendación

La explicabilidad en los SR es una de las áreas más relevantes, tanto en investigación como en entornos empresariales, para mejorar la interacción entre los usuarios y los sistemas

algorítmicos. Un sistema de recomendación explicable no solo busca ofrecer recomendaciones útiles, sino también proporcionar justificación para las decisiones tomadas por el modelo. Esto incrementa la confianza del usuario, facilita la comprensión del sistema y mejora la transparencia [16]. En este contexto, los sistemas de explicabilidad visual emergen como una aproximación poderosa que combina representaciones gráficas y modelos algorítmicos. Estas representaciones permiten a los usuarios interpretar rápidamente los motivos detrás de las recomendaciones ofrecidas.

3.2.1 Definición y Clasificación de la Explicabilidad en SR

Existen múltiples clasificaciones de explicabilidad en sistemas de recomendación, siendo la más popular la de [15], donde la explicabilidad en sistemas de recomendación puede clasificarse dependiendo de dos dimensiones: 1) la fuente de información, o estilo de visualización de las explicaciones, y 2) el propio modelo

Cada una de estas dimensiones puede a su vez separarse en diferentes categorías, en cuanto al estilo de las explicaciones. En este trabajo nuestro interés se centra en las explicaciones visuales, lo que implica el uso de elementos visuales, como imágenes, para hacer las recomendaciones más precisas e intuitivas. Estos modelos utilizan un “idioma” de visualización para representar las explicaciones del sistema [37], siendo los más comunes el diagrama de nodos enlazados, como en [38] y [39] y el diagrama de barras, por ejemplo en la herramienta “Tagspalnation” [40]. Estas formas de representación son intuitivas, aunque no siempre son relevantes para los usuarios. Para abordar este problema, surgen modelos de explicabilidad visual que utilizan imágenes de los ítems, generalmente fotografías, tomadas por otros usuarios, para justificar y explicar las recomendaciones, teniendo en cuenta que una imagen del propio ítem recomendado sí es relevante para el usuario. Por lo tanto, el objetivo es que las imágenes seleccionadas como explicación sean lo más relevantes posibles para el usuario, modelando la selección de imágenes relevantes para el usuario como una tarea de ranking.

3.2.2 Ranking de imágenes en explicabilidad visual

La tarea de ranking de imágenes tiene como objetivo seleccionar la imagen que mejor representa los gustos de un usuario o grupo de usuarios con respecto a un ítem. Esto implica una combinación de aprendizaje de preferencias y análisis de características visuales, manteniendo un equilibrio entre personalización y explicabilidad.

Representación del Problema

Consideremos un conjunto de usuarios U y un conjunto de ítems I . En un sistema de recomendación colaborativo tradicional, las interacciones entre usuarios e ítems se representan

mediante pares etiquetados:

$$(u, i) \in \{0, 1\},$$

donde $u \in U$ es un usuario, $i \in I$ es un ítem, y las etiquetas 1 y 0 denotan si al usuario u le gusta o no el ítem i , respectivamente. En este enfoque, introducimos un conjunto de fotografías tomadas por los usuarios, definiendo las siguientes colecciones:

- $photos(u)$: fotografías tomadas por el usuario $u \in U$
- $photos(i)$: fotografías del ítem $i \in I$
- $photos(u, i) = photos(u) \cap photos(i)$: fotografías tomadas por u relacionadas con i

Clasificación de Fotografías

Se plantea un problema de clasificación binaria para etiquetar las fotografías según su autoría. Para cada par (u, p) donde $u \in U$ y $p \in photos(i)$ la etiqueta se define como:

$$(u, p) = \begin{cases} 1, & p \in photos(u), \\ 0, & p \notin photos(u). \end{cases}$$

La probabilidad de que p sea tomada por u se denota como $\Pr(u, p)$.

Selección de Imágenes Representativas

El objetivo es seleccionar la fotografía $p^* \in photos(i)$ que mejor represente los gustos de un usuario u , maximizando $\Pr(u, p)$:

$$p^* = \arg \max_{p \in photos(i)} \Pr(u, p).$$

Este modelo se puede aplicar para identificar fotografías tomadas por otros usuarios que probablemente también sean del interés de u .

Esta técnica, al permitir predecir imágenes relevantes para cada usuario, abre las puertas a nuevas posibilidades para la explicabilidad visual, utilizando imágenes de otros usuarios para justificar recomendaciones. El primer modelo en implementar esta técnica para generar recomendaciones personalizadas fue [Visual Bayesian Personalized Ranking \(VBPR\)](#) [41] en 2015, utilizando [Convolutional Neural Networks \(CNN\)](#) para aprender representaciones visuales de ítems, y combinándolos con factores latentes tradicionales en un marco de factorización de matrices. Desde entonces han aparecido varios modelos siguiendo los pasos de [VBPR](#), y en nuestro caso vamos a centrarnos en los que componen el Estado del Arte, que son además los modelos que utilizamos en nuestros experimentos, [Explaining Likings Visually \(ELVis\)](#) [8],

Matrix Factorization ELVis (MF-ELVis) [9], y Bayesian Ranking of Images for Explainability (BRIE) [10].

3.2.3 Explaining Likings Visually (ELVis) [8]

Este modelo de explicabilidad visual hace uso de una red neuronal para calcular $\Pr(u, p)$. La entrada del modelo es cada par (u, p) , codificados de manera específica:

- Usuario u en codificación *one-hot* y mapeado a un vector de 256 dimensiones
- Imagen p codificada a través de una Red Neuronal Convolutiva CNN, en concreto, la base convolutiva del modelo Inception-ResNet-v2 [42], con pesos pre-entrenados con ImageNet [43], transformando cada imagen en un vector de 1536 dimensiones, y siendo estos vectores una vez más transformados por una capa Fully Connected (FC) a vectores de 256 elementos.

Tras esto, cada par de vectores (u, p) es concatenado y procesado por un Perceptrón Multi-capas (MLP), aprendiendo una función no lineal que relaciona con cada usuario, sus imágenes, con una función de activación sigmoide para producir como salida una probabilidad entre 0 y 1. La arquitectura y topología del modelo ELVis se puede ver de forma gráfica en la Figura 3.1.

Este modelo presenta muy buenos resultados, aunque el uso de arquitecturas de Redes Neuronales Profundas ocasiona un coste importante de recursos y emisiones [9], implicando un gran problema de sostenibilidad que modelos posteriores intentarán resolver.

3.2.4 Matrix Factorization ELVis (MF-ELVis) [9]

Este modelo, posterior a ELVis, evita utilizar Aprendizaje Profundo por cuestiones de sostenibilidad, reemplazando este modelo interno por Factorización Matricial, técnica explicada previamente en la Sección 3.1.2. Las entradas de este modelo son las mismas que en ELVis, excepto con vectores de 1024 dimensiones. Después de esto, el modelo reemplaza las costosas capas de la Red de Aprendizaje Profundo por la Factorización Matricial entre el usuario u y sus imágenes p , computando un producto escalar entre los vectores resultantes del procesamiento de estos elementos. La función sigmoide final se mantiene, para, de nuevo, obtener los resultados como probabilidad entre 0 y 1 para cualquier imagen p de pertenecer a un usuario u . La arquitectura y topología del modelo MF-ELVis se puede ver en la Figura 3.2 En la comparativa realizada en el mismo trabajo donde se presenta el modelo se puede ver que obtiene resultados competitivos, aunque ligeramente inferiores que ELVis, con un quinto de las emisiones de CO₂, medidas con el sistema “CodeCarbon” [6], y tiempos de entrenamiento hasta 4 veces más rápidos, requiriendo un entrenamiento de 15 épocas frente a las 100 de ELVis, al

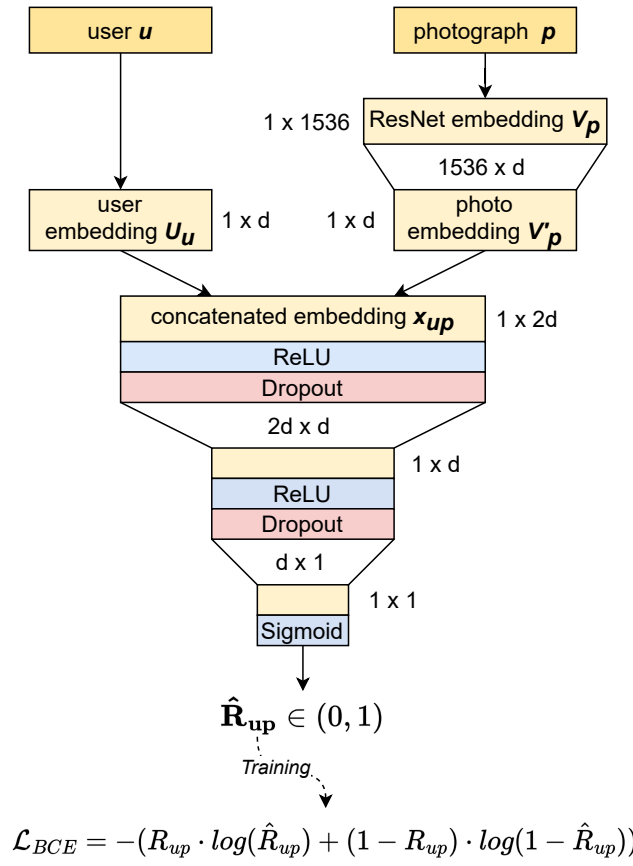


Figura 3.1: Arquitectura y topología del modelo ELVis. Obtenida de [10].

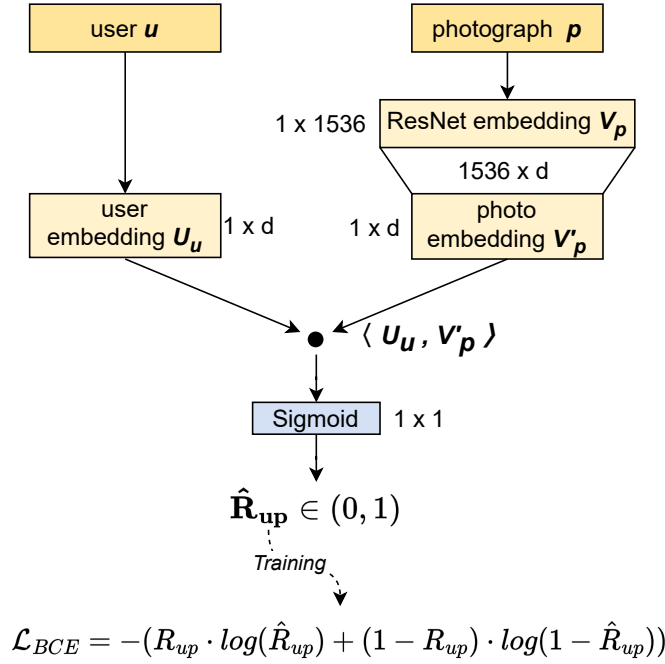


Figura 3.2: Arquitectura y topología del modelo MF-ELVis. Obtenida de [10].

tener una arquitectura mucho más simple. Este modelo indica un paso hacia la explicabilidad visual sostenible, aumentando considerablemente la sostenibilidad a costa de una pequeña reducción en el rendimiento.

3.2.5 Bayesian Ranking of Images for Explainability (BRIE) [10]

Este modelo es un sucesor directo de MF-ELVis y actualmente es el Estado del Arte en explicabilidad visual a través de contenido generado por usuarios. A diferencia de ELVis y MF-ELVis, que emplean clasificación binaria como tarea auxiliar para aproximar un problema de ranking, BRIE adopta el algoritmo de Bayesian Pairwise Ranking (BPR) [44]. Este enfoque optimiza directamente una función de ranking que maximiza la probabilidad de ordenar correctamente las instancias positivas (buenas explicaciones) por encima de las negativas (malas explicaciones).

En este contexto:

- Las instancias positivas se consideran las imágenes subidas por el usuario u , ya que son representativas de los aspectos visuales que este utiliza para justificar sus opiniones.
- Las instancias negativas son imágenes que no han sido subidas por u . Aunque esto simplifica el problema, ignora posibles casos donde imágenes que no son del usuario compartan características explicativas con las suyas. Este desafío se identifica como

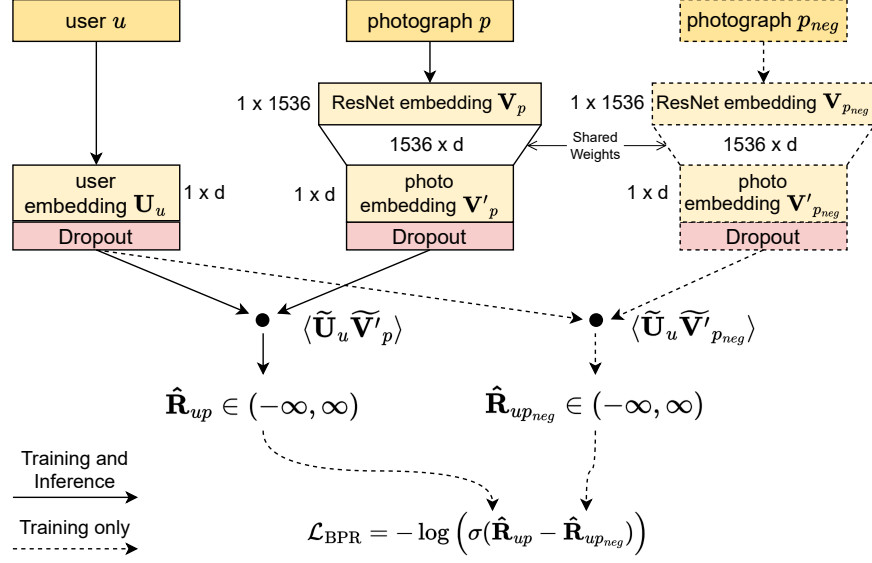


Figura 3.3: Arquitectura y topología del modelo BRIE. Obtenida de [10].

un problema de Aprendizaje Positivo sin Etiquetas, más comúnmente conocido como **Positive Unlabelled Learning (PU Learning)**.

El conjunto de entrenamiento extendido $\mathcal{D}^+$ de BRIE se genera dinámicamente al inicio de cada época, seleccionando uniformemente una imagen negativa p_{neg} para cada par (u, i, p) en el conjunto de interacciones históricas $\mathcal{D}$. Esto evita almacenar muestras negativas pre-seleccionadas, como ocurre en ELVis y MF-ELVis, permitiendo un proceso más eficiente. La función de pérdida de BPR utilizada se define como:

$$\mathcal{L}_{BPR} = - \sum_{(u, i, p, p_{neg}) \in \mathcal{D}^+} \log \left(\sigma(\hat{R}_{up} - \hat{R}_{up_{neg}}) \right),$$

donde σ representa la función sigmoide, $\hat{R}_{up}$ es la probabilidad predicha de que p sea una buena explicación para u , y p_{neg} es una imagen negativa.

Durante el entrenamiento, para mitigar problemas de sobreajuste derivados del uso de BPR, se aplica regularización por *dropout* a las representaciones latentes, produciendo los vectores modificados $\tilde{\mathbf{U}}_u$ y $\tilde{\mathbf{V}}'_p$. La puntuación de autoría predicha, $\hat{R}_{up}$, para el par (u, p) se calcula como:

$$\hat{R}_{up} = \langle \tilde{\mathbf{U}}_u, \tilde{\mathbf{V}}'_p \rangle,$$

donde $\langle \cdot \rangle$ denota el producto escalar. A diferencia de sus predecesores, BRIE no emplea una activación sigmoide, ya que no busca limitar $\hat{R}_{up}$ al rango $[0, 1]$. La arquitectura y topología del modelo BRIE se puede ver en la Figura 3.3. Con estas optimizaciones, BRIE es capaz de superar

a los modelos [ELVis](#) y [MF-ELVis](#) en todos los conjuntos de datos para todas las métricas de rendimiento utilizadas. Además consigue recortar las emisiones de CO2 de la fase de inferencia del modelo, según mediciones con *CodeCarbon* [6], en un 75% en comparación con [ELVis](#) y en un 50% en comparación con [MF-ELVis](#).

3.3 Modelos de *Embedding* de Imágenes

Una parte importante del rendimiento de los modelos de explicabilidad visual es la vectorización de las características latentes de las imágenes. Para lograr esta vectorización se utilizan los llamados modelos de *embedding*, en este caso, de imágenes. Estos modelos permiten transformar imágenes en representaciones vectoriales de alta dimensión, que capturan la esencia o significado, llamado características latentes, de la imagen, permitiendo a modelos de [AA](#) entender la similitud o diferencia entre imágenes.

En el trabajo original que describe [ELVis](#) [8] y en los modelos que hemos descrito como evoluciones (MF-ELVis y BRIE) se ha utilizado el mismo modelo de *embedding* de imágenes, el modelo Inception-ResNet-v2 [42]. El modelo es en si la mezcla de dos conceptos de modelos diferentes, Inception [45] y ResNet [46], con esta síntesis produciendo resultados Estado del Arte de su época en clasificación de imágenes. Su diseño técnico se basa en una estructura compleja que integra módulos de procesamiento paralelo con conexiones residuales, con 164 capas y 55.8 millones de parámetros. Aún siendo un modelo de clasificación de imágenes, unos pequeños ajustes hacen que tanto este como muchos otros modelos de clasificación funcionen como extractores de características, únicamente hace falta eliminar la capa final de “Global Average Pooling”, que se encarga de transformar las características que extrae el modelo de las imágenes en un vector de una dimensión para clasificar la imagen. Los resultados sin esta capa final son las características latentes de la imagen introducida, es decir, la representación vectorizada.

3.4 Técnicas de mejora de calidad de los Datos

Una de las maneras de aumentar el rendimiento de los modelos de [AA](#) es mejorando la calidad de los datos de entrada (entrenamiento y validación), esta técnica no se ha aplicado previamente para el problema de la explicabilidad visual, se podría aplicar a las imágenes del conjunto de datos, facilitando la tarea de ranking de imágenes de usuarios al poder obtener más información del conjunto de imágenes, o de cada imagen individualmente. Mejorar la calidad de los Datos es un concepto abierto, por ello, vamos a exponer los diferentes conceptos útiles para producir datos de mayor calidad.

3.4.1 Aprendizaje Positivo y Sin Etiquetas

Los modelos utilizados en este trabajo, presentados en la Sección 3, comparten una característica común: abordan la explicabilidad visual como una tarea de aprendizaje basada en la clasificación por ranking de imágenes. Sin embargo, esto plantea un desafío particular, ya que únicamente se dispone de ejemplos positivos confiables, representados por las imágenes asociadas al usuario correspondiente. Sin embargo, no contamos con ejemplos claramente etiquetados como negativos, ya que las imágenes de otros usuarios, aunque no sean del usuario objetivo, no garantizan ser verdaderos negativos en términos de similitud o afinidad con sus preferencias. Este sesgo en la disponibilidad de datos etiquetados limita la aplicabilidad directa de técnicas tradicionales de clasificación, que requieren tanto ejemplos positivos como negativos bien definidos. Para abordar este desafío, una solución es utilizar un enfoque basado en [Positive Unlabelled Learning \(PU Learning\)](#) [47] o Aprendizaje Positivo y Sin Etiquetas, una técnica que permite tratar los datos no etiquetados como una mezcla de ejemplos negativos potenciales y ruido. El [PU Learning](#) es una técnica de clasificación binaria utilizada en situaciones donde el conjunto de datos de entrenamiento contiene ejemplos positivos etiquetados y ejemplos no etiquetados, pero no se dispone de ejemplos negativos etiquetados. En este escenario, se asume que los ejemplos no etiquetados pueden pertenecer tanto a la clase positiva como a la negativa. Este paradigma ha ganado atención en aplicaciones como diagnósticos médicos, publicidad personalizada y la completación de bases de conocimiento, donde los datos negativos suelen estar ausentes o son difíciles de identificar [47].

Fundamentos técnicos del [PU Learning](#)

En el aprendizaje supervisado tradicional, el objetivo es entrenar un modelo que pueda distinguir entre dos clases (positiva y negativa) utilizando un conjunto de datos completamente etiquetado. En contraste, [PU Learning](#) trabaja bajo las siguientes condiciones:

- **Presencia de datos no etiquetados:** Los datos no etiquetados incluyen ejemplos positivos y negativos, pero no se tiene información directa sobre su clase.
- **Mecanismo de etiquetado probabilístico:** Los ejemplos positivos etiquetados se seleccionan de acuerdo con una probabilidad conocida como propensity score.

La representación formal de un conjunto de datos PU incluye tripletas (x, y, s) donde x representa los atributos, y la clase (positiva o negativa), y z indica si la muestra está etiquetada.

Clasificación de técnicas de [PU Learning](#)

Hay tres categorías principales en las que se dividen las técnicas de [PU Learning](#):

- **Técnicas de dos pasos:** Técnicas que combinan la identificación de ejemplos negativos confiables con algoritmos de aprendizaje supervisado
- **Aprendizaje sesgado:** Trata ejemplos no etiquetados como negativos con ruido en las etiquetas, ajustando los pesos o penalizaciones para reflejar la incertidumbre.
- **Incorporación del *priori de la clase*:** Modifica métodos de aprendizaje tradicionales para incluir el conocimiento previo sobre la proporción de clases positivas.

De estas técnicas, la más fácilmente aplicable a nuestra problemática es la técnica de dos pasos, siendo nuestro objetivo encontrar negativos confiables. Una de las aproximaciones más destacadas en este contexto es el método **Rocchio**. Este método utiliza representaciones vectoriales para construir prototipos de clases basado en las medias ponderadas de los vectores de atributos de las clases positiva y negativa, el prototipo positivo se calcula a partir de los datos etiquetados como positivos, mientras que el prototipo negativo se construye utilizando ejemplos no etiquetados que sean similares entre sí pero diferentes de los positivos. La clasificación se realiza comparando la distancia de los ejemplos no etiquetados a cada prototipo. Ejemplos más cercanos al prototipo negativo se consideran negativos confiables. Con esta base teórica, desarrollamos diferentes aproximaciones, detalladas en el Capítulo 4.

Aumentación de los Datos

La idea principal detrás de la aumentación de datos es introducir variaciones en las imágenes que sean coherentes con el dominio del problema, manteniendo las características esenciales necesarias para la tarea de aprendizaje. Entre las transformaciones más comunes se incluyen [48]:

- **Transformaciones geométricas:** Rotaciones, traslaciones, escalado y reflejos. Estas operaciones permiten que el modelo sea más robusto frente a variaciones espaciales y angulares.
- **Ajustes de color y brillo:** Modificaciones en la intensidad de los colores, el contraste y el brillo, lo que permite al modelo ser menos sensible a condiciones de iluminación variables.
- **Adición de ruido:** Introducir ruido gaussiano o de otro tipo para simular imperfecciones comunes en las imágenes reales.
- **Recortes aleatorios:** Seleccionar y procesar subregiones de la imagen para fomentar la detección de características relevantes en diferentes partes de la misma.

La aumentación de datos no solo es útil para aumentar el tamaño del conjunto de datos, sino también para mejorar la capacidad de generalización del modelo al exponerlo a una mayor diversidad de ejemplos. Por ejemplo, en tareas de clasificación de imágenes, un modelo entrenado con datos aumentados suele ser más robusto frente a variaciones que podrían no estar presentes en el conjunto de datos original. Estas mejoras posiblemente se puedan trasladar a la tarea de ranking de imágenes, haciendo que los modelos tengan mejores capacidades para reconocer imágenes del usuario no vistas anteriormente.

Inteligencia Artificial Generativa

La Inteligencia Artificial Generativa es un conjunto de técnicas y modelos diseñados para crear datos nuevos y realistas a partir de distribuciones de datos existentes, ya sea texto, imágenes, audio o video [49]. En el caso específico de imágenes, la IA generativa ha demostrado ser una herramienta poderosa para abordar desafíos como el desbalance de datos, la creación de datos sintéticos y la mejora de la diversidad en los conjuntos de datos [48, 50]. Esto puede llevarse a cabo de múltiples maneras, aunque las más comunes son 2, crear imágenes derivadas de otras imágenes, y crear imágenes a partir de texto. Creando imágenes sintéticas, el resultado es similar al de la aumentación de datos, siendo considerado incluso como una forma avanzada de esta técnica [48], con la capacidad de añadir nueva información al conjunto de datos, duplicando el “concepto” detrás de una imagen en vez de esta misma.

Técnicas propuestas

AHORA que hemos presentado los conceptos y modelos en los que se apoya nuestro trabajo, podemos continuar presentando las soluciones a la problemática que es mejorar el rendimiento de estos modelos de explicabilidad visual en [SR](#). Para ello, en este capítulo explicaremos uno a uno los métodos y técnicas diseñados para mejorar el rendimiento de los modelos de explicabilidad visual explicados en el Capítulo 3. Con los datos presentados, podemos comenzar a explicar los métodos propuestos para mejorar el rendimiento de los modelos. Los fundamentos técnicos de los métodos que presentaremos a continuación se pueden encontrar en el Capítulo 3. El objetivo de todas nuestras técnicas es aumentar la calidad de los datos de entrada (entrenamiento y validación); esto va desde mejorar la calidad de los ejemplos negativos para cada usuario, eligiendo como negativos imágenes no solo de otros usuarios, si no lo más diferentes posibles a las imágenes del usuario, hasta mejorar la calidad de los vectores de características latentes de las imágenes. Nuestros métodos buscan, además, que el intercambio entre sostenibilidad y rendimiento sea lo más favorable posible.

4.1 Aprendizaje Positivo y Sin Etiquetas

Hasta el momento, para trabajos anteriores con los mismos conjuntos de datos utilizados en este trabajo, los ejemplos negativos para cada usuario fueron seleccionados aleatoriamente entre las imágenes del resto de usuarios. Realmente, no se puede asegurar que una imagen de otro usuario sea un negativo fiable, ya que podría reflejar perfectamente los gustos del usuario a recomendar. Debido a esto, podemos formular el problema como un problema de Aprendizaje Positivo y Sin Etiquetas o [PU Learning](#), en el que los únicos ejemplos etiquetados son las imágenes de cada usuario para sí mismo como positivos, y el resto de ejemplos siendo no etiquetados. Para obtener negativos, en lugar de realizar una selección aleatoria, que puede etiquetar erróneamente como negativos imágenes de otros usuarios que comparten características explicativas útiles para el usuario objetivo, proponemos dos métodos de [PU Learning](#)

diferentes:

4.1.1 Selección de negativos mediante similitud de imágenes

Para cada usuario, este método identifica imágenes de otros usuarios que difieren significativamente de las del usuario y las utiliza como ejemplos negativos fiables, evitando introducir ruido en las etiquetas durante el entrenamiento. Esta técnica fue primeramente publicada para el modelo ELVis [11], nuestros objetivos son adaptar la técnica para conseguir una funcionalidad más general y poder aplicarla al resto de modelos de explicabilidad visual utilizados, MF-ELVis y BRIE. Este enfoque fue diseñado bajo el supuesto *Select Completely At Random* (SCAR) [47], que postula que el conjunto de ejemplos positivos etiquetados forma una muestra *independiente e idénticamente distribuida* (i.i.d.) de la distribución positiva. En nuestro caso particular, esto implica asumir, para cada usuario, 1) uniformidad y 2) separabilidad en el conjunto no etiquetado (es decir, imágenes subidas por otros usuarios). Esto significa que, para cualquier usuario dado:

1. Las imágenes que son buenas explicaciones para el usuario tienen características similares a las del usuario.
2. Las imágenes que no son buenas explicaciones para el usuario tienen características disímiles a las del usuario.

Para el primer paso del paradigma en dos pasos, que consiste en la selección de estos negativos fiables, diseñamos un enfoque basado en la personalización usuario a usuario mediante un clasificador Rocchio [51]. En lugar de definir un criterio global para la selección de negativos fiables, este criterio es particular para cada usuario, basado en sus preferencias explicativas. Este proceso, ilustrado en la Figura 4.1, funciona de la siguiente manera para cada usuario u :

1. Se encuentra un prototipo de los ejemplos positivos conocidos de u (es decir, las imágenes originalmente subidas por u) calculando el centróide C_u de las imágenes usando las representaciones latentes obtenidas con InceptionResNetV2 de tamaño $d = 1536$:

$$C_u = \frac{1}{|P_u|} \sum_{p \in P_u} V'_p$$

donde P_u es el conjunto de imágenes de u , y V'_p es la representación latente de una imagen p . Este centróide C_u forma una representación global de las imágenes de u , es decir, un prototipo de lo que constituye una buena explicación para u .

2. Se define un criterio de similitud que actúa como umbral para la selección de negativos fiables. Este umbral se define como el percentil 10 (P_{10}) de las similitudes entre las

imágenes del usuario y su centróide C_u , debido a sus buenas propiedades de filtrado de valores atípicos en trabajos existentes de [PU Learning](#) con clasificadores Rocchio. La métrica de similitud es una métrica genérica, definida como:

$$SC(V_p, V_{p'}) = f(V_p, V_{p'})$$

donde (p, p') es cualquier par de imágenes. Una $SC(V_p, V_{p'}) = 1$ indica máxima similitud, y valores cercanos a -1 indican disimilitud.

3. Antes de iniciar el entrenamiento, se seleccionan ejemplos negativos (imágenes que no son útiles como explicaciones para el usuario) para cada usuario. Para asegurar una comparación justa, el esquema de selección sigue el método del Estado del Arte ELVis [8]: para cada imagen del usuario (es decir, un ejemplo positivo (u, i, p)), se seleccionan 20 imágenes, de las cuales 10 provienen del mismo restaurante i (formando ejemplos negativos (u, i, p_{neg})) y las otras 10 de restaurantes diferentes (u, i', p_{neg}) . Nuestro enfoque de [PU Learning](#) añade un criterio estricto de selección para la fiabilidad de los ejemplos negativos: todas las imágenes negativas seleccionadas p_{neg} deben cumplir que:

$$SC(C_u, V_{p_{neg}}) \leq P_{10}$$

asegurando así que son disímiles a las del usuario, evitando el ruido en las etiquetas causado por la selección aleatoria utilizada en el Estado del Arte.

Una vez completada la selección de ejemplos negativos fiables, se utiliza un conjunto completo de entrenamiento con ejemplos positivos (imágenes subidas originalmente por los usuarios, es decir, buenas explicaciones de recomendación para ellos) y ejemplos negativos fiables (imágenes que es poco probable que sean subidas por los usuarios con los que se han emparejado, es decir, malas explicaciones de recomendación para ellos). Este proceso de [PU Learning](#) es agnóstico al modelo y podría utilizarse con otros modelos de explicabilidad visual diferentes a los usados en este trabajo.

4.1.2 Selección de nuevos positivos mediante similitud de usuarios

En [PU Learning](#), además de mejorar la selección de ejemplos negativos, también es posible potenciar el rendimiento del modelo mediante la adición de nuevos ejemplos positivos [47], capturando mejor la estructura latente de las preferencias de cada usuario. Para lograr esto, diseñamos un enfoque basado en la similitud entre usuarios que permite ampliar de manera controlada el conjunto de positivos de cada usuario. Este procedimiento sigue los siguientes pasos:

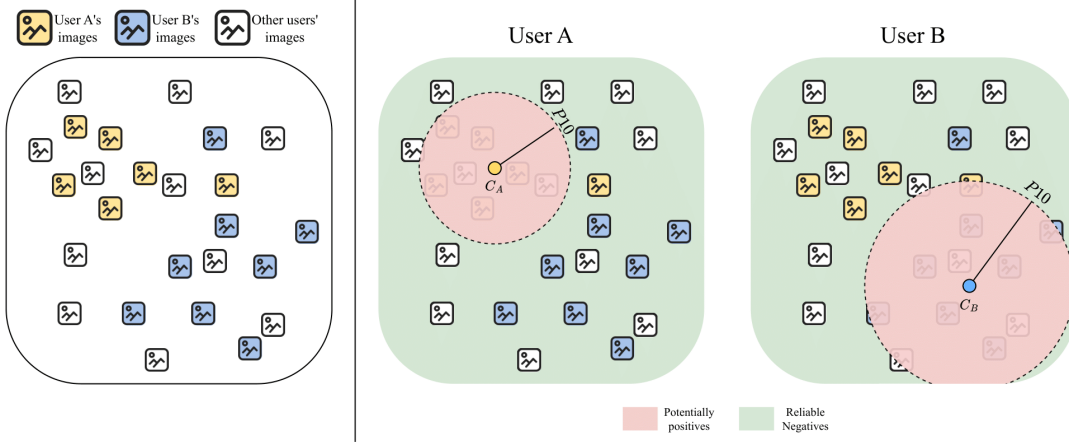


Figura 4.1: Enfoque de **PU Learning** personalizado para seleccionar de forma fiable imágenes negativas (malas explicaciones) para cada usuario en el contexto de la explicación de la recomendación. En este ejemplo, se muestra la frontera de decisión personalizada para la selección de ejemplos negativos fiables para dos usuarios A y B con diferentes preferencias de explicación. Obtenida del artículo original de la técnica para ELVis [11].

1. Al finalizar cada época de entrenamiento, se actualiza el prototipo de cada usuario C_u utilizando el promedio de las representaciones latentes de sus imágenes positivas, de manera análoga a lo descrito en la Sección previa:

$$C_u = \frac{1}{|P_u|} \sum_{p \in P_u} V_p'.$$

2. Se define una métrica de similitud entre los prototipos de usuarios C_u y $C_{u'}$:

$$SC(C_u, C_{u'}) = f(C_u, C_{u'})$$

3. Para cada usuario u , se selecciona un conjunto de k usuarios más similares según la métrica anterior.
4. De cada usuario u' seleccionado, se añade una imagen positiva a la colección de imágenes positivas de u , asegurando que los nuevos ejemplos añadidos sean representativos de su perfil explicativo.

Este procedimiento permite enriquecer el conjunto de positivos de cada usuario con ejemplos relevantes obtenidos de usuarios con perfiles explicativos similares. Trabajos previos han demostrado que enfoques de este tipo pueden mejorar la calidad de las explicaciones generadas por los modelos de recomendación [52].

4.2 Aumentación de los Datos

Debido a la dispersión presente en los conjuntos de datos utilizados (50% de imágenes provienen de usuarios con menos de 5 imágenes entre sus reseñas, siendo este grupo de usuarios un 90% del total de usuarios), proponemos que un método que incremente la cantidad de información presente para estos usuarios menos representados podría aumentar el rendimiento global de los modelos en su tarea de *ránking* de imágenes. Esto es de especial importancia por el problema de “comienzo en frío”, donde los usuarios menos representados en los conjuntos de datos tendrían resultados peores que los usuarios con mayor representación. Una técnica comunmente utilizada en problemas de Aprendizaje Automático (AA) con imágenes es la de la Aumentación de los Datos, una serie de técnicas con como objetivo principal mejorar la capacidad de generalización de los modelos de AA al ampliar el conjunto de datos, reduciendo los sesgos y el riesgo de sobreentrenamiento (*overfitting*) [48]. A diferencia de la simple duplicación de datos, la aumentación introduce variaciones que pueden hacer que los modelos sean más robustos frente a cambios en la distribución de los datos reales. Utilizando la Aumentación de los Datos, presentamos dos diferentes soluciones para mejorar el rendimiento de los modelos de explicabilidad visual: Aumentación de los Datos por transformación de imágenes, y Aumentación de los datos por Inteligencia Artificial Generativa.

4.2.1 Aumentación de los Datos por transformación de imágenes

La forma clásica de llevar a cabo la aumentación de datos consiste en aplicar una serie de transformaciones para generar nuevos datos a partir de muestras existentes. En el ámbito de imágenes, las técnicas de aumentación abarcan desde ajustes simples en el dominio del color (p. ej., modificar brillo, contraste o saturación) hasta alteraciones geométricas (como rotaciones, reflejos o desplazamientos). Estudios recientes [48] destacan que no todas las transformaciones aportan el mismo beneficio: entre las opciones básicas, el recorte aleatorio sobresale por su eficacia, ya que fuerza al modelo a reconocer características relevantes en distintas regiones de la imagen, evitando la dependencia excesiva de elementos periféricos.

Para evaluar el impacto de la aumentación en la explicabilidad visual, diseñamos un conjunto de técnicas avanzadas con mejor rendimiento, según la literatura, que las transformaciones básicas como rotación de imagen o cambios de color [48]:

- *Recorte aleatorio*: Enfatiza detalles locales al extraer subregiones de la imagen.
- *Deformación geométrica*: Modifica la perspectiva para imitar capturas desde ángulos no convencionales.
- *Ruido gaussiano*: Añade borrosidad controlada para mejorar tolerancia a imperfecciones o distorsiones.

La Figura 4.2 muestra ejemplos de cada una de estas transformaciones aplicadas a una imagen original.

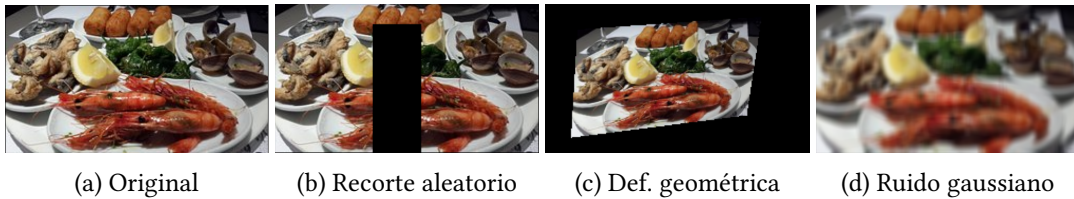


Figura 4.2: Transformaciones utilizadas sobre las imágenes de los conjuntos de datos como aumentación de los datos.

Aplicamos estas transformaciones únicamente a usuarios con menos de cinco imágenes en sus reseñas, por ser los usuarios más comunes y en los que la aumentación de sus imágenes debería tener un efecto mayor. De este modo, buscamos enriquecer los datos sin comprometer la capacidad del modelo para generalizar de manera efectiva [48].

4.2.2 Inteligencia Artificial Generativa

La Inteligencia Artificial Generativa engloba un gran número de técnicas y modelos de [AA](#) capaces de crear datos nuevos a partir de datos preexistentes. Estas técnicas se han aplicado a una amplia variedad de dominios, incluidos texto, imágenes, audio y video [49]. Nuestro interés se centra en la generación de imágenes, con la idea de generar imágenes para estos usuarios con menor representación. Para esto, obtenemos los textos de las reseñas reales de los usuarios de este conjunto deseado, y los transformamos en consultas o “prompts” para un modelo de [IA](#) generativa de la siguiente manera:

*Imagen fotorealista, tomada con la cámara de un móvil, generada a partir de la siguiente reseña de un restaurante: **texto reseña***

Ejemplo de consulta:

Imagen fotorealista, tomada con la cámara de un móvil, generada a partir de la siguiente reseña de un restaurante: Aunque a primera vista pueda parecer un sitio no muy acogedor, la verdad es que el Mesón Sancho, con unas cuantas décadas de tradición, hace las delicias de los que gustan del buen comer, y sin duda el que prueba, repite. Con una cocina tradicional, tiene especialidad en carnes, entre las que se puede degustar un excelente chuletón a la piedra con toque argentino, chuletillos de cordero o caza en determinadas épocas del año. El pescado también es espectacular. No sólo es la carne y el pescado, creo que también es su toque especial y el cariño que ponen en su cocina.



Figura 4.3: Imagen obtenida con la consulta 4.2.2 con StableDiffusion3.5.

Para generar estas imágenes, hace falta escoger entre los múltiples modelos de generación de imágenes a partir de consultas de texto disponibles públicamente, en concreto, un modelo pre-entrenado, ya que aunque poseemos los recursos computacionales necesarios, al contar con los servidores del [Centro de Investigación en TIC \(CITIC\)](#), nuestros conjuntos de datos no son suficientes como para pre-entrenar un modelo desde cero, y la tarea de recopilar nuevas imágenes se sale fuera de los parámetros marcados en este trabajo. Se barajaron varias alternativas, primero, por su importancia y sus resultados Estado del Arte, los modelos abiertos de StabilityAI, StableDiffusion.[53]

Un ejemplo de imagen obtenida con la consulta mostrada anteriormente con el último modelo de StableDiffusion, StableDiffusion3.5 (SD3.5) se puede observar en la Figura 4.3.

Por desgracia, aunque su uso es viable en una alternativa comercial, los costes de generación de imágenes con su SD3.5 pre-entrenado, que ascienden a 10€ por cada 150 imágenes a través de la API de StableAI, sobrepasa los límites de este trabajo, esto se debe a las enormes dimensiones del modelo, de 2,5 mil millones de parámetros para la versión Medium, requiriendo recursos computacionales que, aunque tenemos disponibles, ponen en riesgo la sostenibilidad de la solución. Otra alternativa de StableDiffusion que obtienen muy buenos resultados con un menor coste computacional es StableDiffusion1.5, con un número menor de parámetros (983 millones). Un ejemplo de imagen obtenida con la consulta anteriormente mostrada puede verse en la Figura 4.4.

Este modelo sigue teniendo un problema importante, en vez de un límite económico, sobrepasa el límite temporal del proyecto. Debido a la velocidad del modelo, los tiempos de generación de imágenes se salen fuera de los marcos de este proyecto, tomando semanas de



Figura 4.4: Imagen obtenida con la consulta 4.2.2 con StableDiffusion1.5.

computación para cualquier conjunto de datos excepto para el conjunto de datos Gijon, por su menor tamaño.

Una manera de acelerar la generación de imágenes de estos modelos es reduciendo el tamaño de las imágenes generadas, algo válido e incluso deseable para nuestro proyecto, siendo las imágenes originales tamaño 150x150; desgraciadamente, al haberse realizado un pre-entrenamiento de los modelos con imágenes de tamaño 512x512 y superior, es virtualmente incapaz de generar imágenes de menor tamaño de manera fiable, con resultados como el de la Figura 4.5, o siendo paradas incorrectamente por el filtro de imágenes no seguras del modelo.

Teniendo dos limitaciones importantes, económica y temporal, el modelo a utilizar debe ser gratuito y rápido.

Por desgracia, ningún modelo de Difusión cumple las condiciones necesarias. Afortunadamente, alejándonos de modelos de difusión, existe el modelo aMUSEd[54] ligero y centrado en la velocidad de generación de imágenes. El modelo aMUSEd, como su nombre indica, es una reproducción de un modelo anterior, [55], aunque utilizando una décima parte de los parámetros para hacer un modelo extremadamente ligero, con 800 millones de parámetros. Este número puede parecer similar al número de parámetros de StableDiffusion1.5, pero la diferencia de arquitectura subyacente, siendo un modelo de Enmascaramiento de Imágenes o *Masked Image Model* (MIM) [56] en vez de de Difusión. Este tipo de modelos entrenan mediante la predicción de regiones ocultas en imágenes, adaptando el concepto de modelos enmascarados en lenguaje natural, como *Bidirectional Encoder Representations from Transformers* (BERT), al dominio de visión artificial. Este mecanismo es mucho más rápido que Difusión, requiriendo menos pasos de inferencia en comparación [55]. Además de esto, los



Figura 4.5: Imagen obtenida con la consulta 4.2.2 con StableDiffusion1.5 al seleccionar un tamaño de 150x150.

creadores de aMUSEd proporcionaron dos versiones pre-entrenadas de su modelo, una de ellas, aMUSEd256, entrenada únicamente con imágenes de tamaño 256x256, un tamaño mucho más cercano al desdado que el que puede producir cualquier otro modelo, lo que hace que sea capaz de generar hasta 2 imágenes por segundo, permitiendo generar todas las imágenes necesarias en tiempos aceptables, y por lo tanto consumiendo muchos menos recursos que otros modelos más lentos, siendo una solución más sostenible. La calidad de las imágenes, como es de esperar, es significativamente menor que con modelos más lentos y de mayor número de parámetros, como se puede ver en la Figura 4.6, pero esto debería tener menor importancia tras la vectorización de las imágenes, un paso previo a la utilización por parte de los modelos de explicabilidad visual. Otros ejemplos de diferentes características de imágenes generadas con el modelo aMUSEd256 se pueden ver en la Figura 4.7, donde se ve claramente que el modelo, aunque generando imágenes poco realistas, es capaz de reflejar los intereses de diferentes usuarios, con algunos más preocupados por la comida, y otros por las vistas y ambientación.

Con este sistema, generamos imágenes con el modelo aMUSEd256 para los usuarios con menos de 5 imágenes en sus reseñas, creando para cada una de sus reseñas 5 imágenes diferentes, utilizando el texto de la reseña para crear la consulta como se explica previamente. Estas imágenes generadas son introducidas al conjunto de datos de entrenamiento como nuevos positivos, para el mismo usuario y restaurante que la reseña original que se utilizó como consulta.

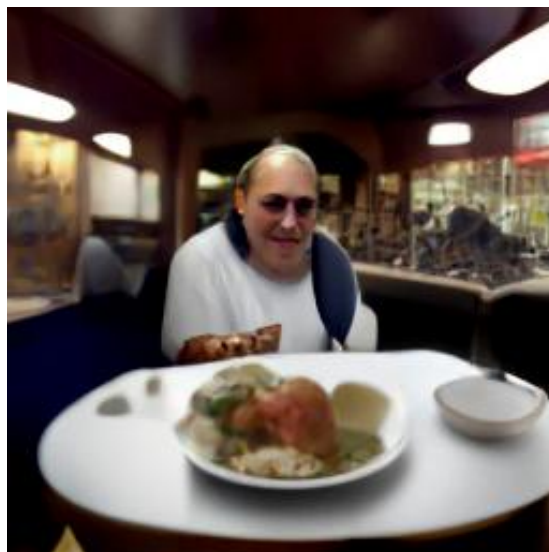


Figura 4.6: Imagen obtenida con la consulta 4.2.2 con aMUSEd256.



Figura 4.7: Imágenes obtenidas con el modelo aMUSEd256 para diferentes consultas creadas a partir de reseñas de usuarios.

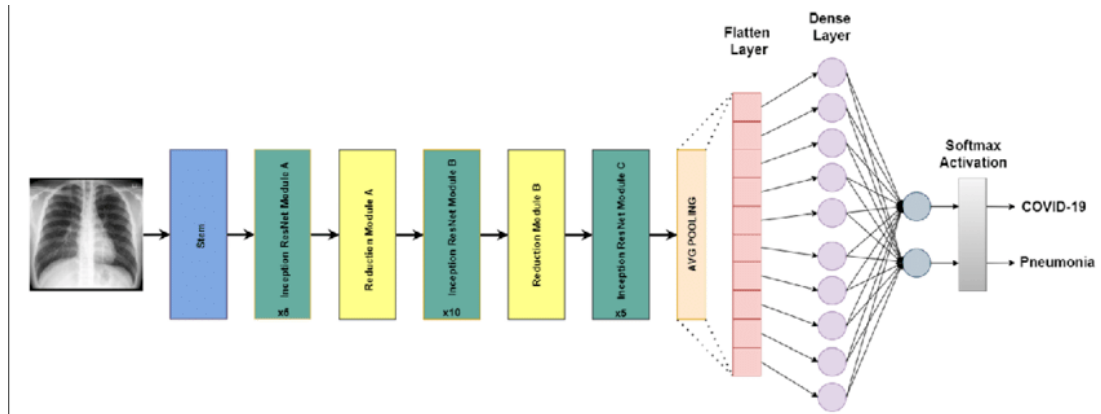


Figura 4.8: Arquitectura del modelo InceptionResnetV2, obtenida de [12].

4.3 Mejora en los *embedding* de imágenes

Una parte importante del rendimiento de los modelos de explicabilidad visual es la vectorización de las características latentes de las imágenes. Como se explica en la Sección 3.3, los modelos de explicabilidad visual usados en este trabajo usan un modelo de *embedding* de imágenes común, el InceptionResnetV2 [42], utilizando como *embedding* latente el estado oculto que se sirve como entrada a la capa final de “Global Average Pooling”. La arquitectura del modelo InceptionResnetV2 puede verse en la Figura 4.8. Sin embargo, este modelo fue publicado oficialmente para su uso en 2017, y el campo de visión artificial ha mejorado a gran velocidad desde entonces. Proponemos entonces, que una manera de obtener mejores resultados es aumentando la calidad de la información que se proporciona de cada imagen, utilizando un modelo más actual del Estado del Arte para obtener *embeddings* que capturen las características latentes de las imágenes con mayor precisión. Además de esto, existen modelos Estado del Arte de tamaños inferiores al propio InceptionResnetV2, lo que podría proporcionar además de una mejora al rendimiento, un aumento de la sostenibilidad y reducción de consumos y emisiones. Para seleccionar un modelo nuevo, utilizamos como referencia una comparativa de rendimiento de modelos de clasificación de imágenes [57] entrenados con ImageNet [43], con el objetivo de aplicar la misma técnica que en InceptionResnetV2 y utilizar los modelos para vectorizar las imágenes de los conjuntos de datos. Hay una gran cantidad de modelos con resultados de Estado del Arte, aunque estamos restringidos a aquellos que podamos obtener pre-entrenados. De todos ellos, nos decidimos por una versión del modelo Contrastive Language-Image Pretraining (CLIP) de OpenAI, en concreto Vision Transformer (ViT) Large 14 ¹ Internamente, el modelo se compone de un codificador de imágenes Vision Transformer (ViT) Large [58] y un codificador de texto, para maximizar de pares (imagen, texto). La

¹https://huggingface.co/timm/vit_large_patch14_clip_336.openai.

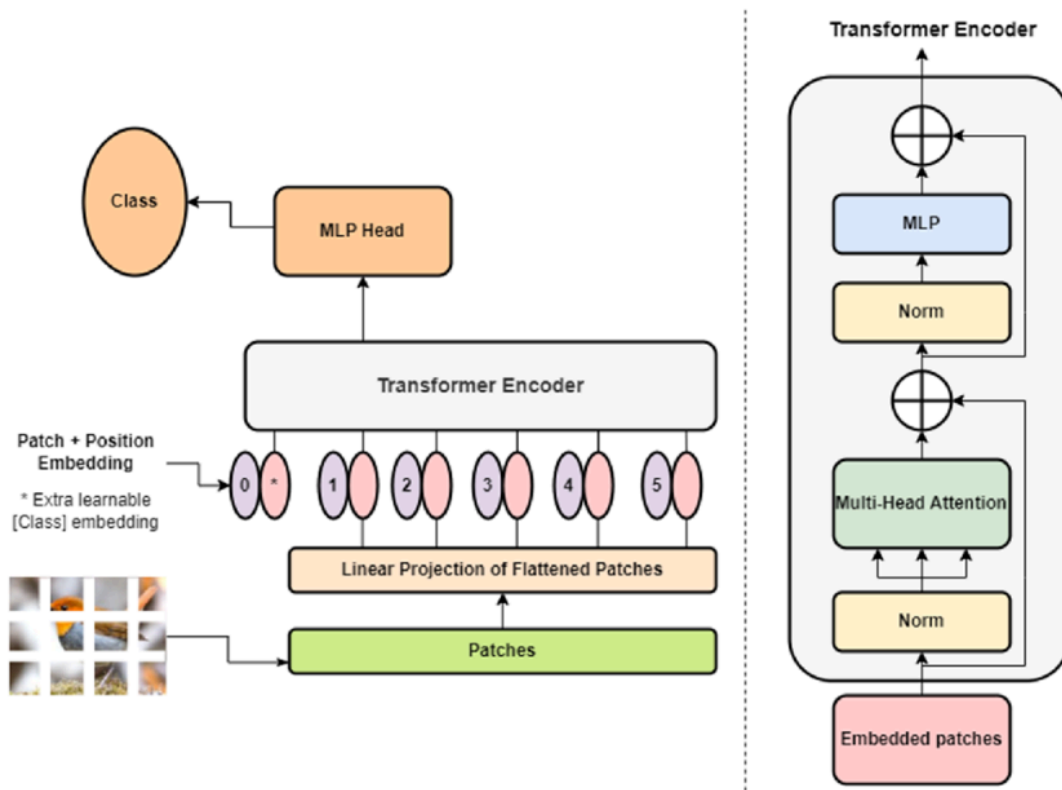


Figura 4.9: Arquitectura del modelo ViT, obtenida de [13].

arquitectura del modelo ViT se puede ver en la Figura 4.9 Este modelo utiliza 400 millones de parámetros, casi 8 veces más que el modelo original InceptionResnetV2, pero es capaz de generar *embeddings* en el mismo tiempo, creando vectores de 768 dimensiones, exactamente la mitad de las 1536 dimensiones de InceptionResnetV2, lo que puede proporcionar una mejora en cuanto a los tiempos de entrenamiento y prueba, reduciendo emisiones, consumos, y creando un modelo más sostenible.

Configuración experimental

ANTES de mostrar resultados, es necesario definir los datos utilizados, y las métricas que usamos para medir el rendimiento de cada una de nuestras aproximaciones, así como los hiper-parámetros utilizados para cada uno de los modelos, este capítulo está dedicado a explicar en detalle la configuración experimental utilizada para obtener los resultados que se observan en el Capítulo 6.

5.1 Datos

Los conjuntos de datos utilizados en nuestros experimentos son los de la publicación original de ELVis [8], y consisten en fotografías subidas por usuarios en el contexto de reseñas de restaurantes en TripAdvisor, una popular plataforma de evaluación de establecimientos de hostelería. La razón del uso de estos conjuntos de datos, más allá de su gran diversidad y tamaño, es la de poder replicar los resultados obtenidos previamente con los modelos utilizados, permitiendo comparar el impacto de cada una de las soluciones propuestas. Para formar los conjuntos de datos, en el trabajo original se utilizaron datos reales de seis ciudades distintas, que de menor a mayor tamaño son: Gijón, Barcelona, Madrid, París, Nueva York, y Londres. Estos datos ofrecen contextos diversos para la evaluación en términos de características socioculturales, escala y comportamientos de usuarios.

Cada muestra en el conjunto de datos bruto consiste en un triplete $(u, i, p) \in D$, donde u representa el ID del usuario, i representa el ID del restaurante, y p la fotografía. La Tabla 5.1 resume las características de cada conjunto de datos utilizado en este trabajo. Debido a su gran tamaño y las limitaciones temporales y de recursos computacionales disponibles, los conjuntos de París, Nueva York, y Londres no fueron usados en nuestro trabajo; con todo, los conjuntos de datos restantes (Gijón, Barcelona, y Madrid) permiten disponer de una evaluación exhaustiva del rendimiento en la tarea. Cabe destacar que el conjunto de datos de Gijón, debido a su pequeño tamaño, presenta gran variación en sus resultados, así que los

resultados mostrados se obtienen de la media de cinco ejecuciones para cada conjunto modelo-técnica. Con el objetivo de garantizar la reproducibilidad y comparabilidad de los resultados, utilizamos la partición de prueba proporcionada por los autores originales, y utilizamos las particiones de entrenamiento y validación como base en nuestros experimentos, gran parte de ellos modifican estos conjuntos con el objetivo de añadir diversidad a los datos y aumentar la cantidad de información que proporcionan. Es de interés explicar la partición original de estos conjuntos de datos:

- Para todos los usuarios con como mínimo $N \geq 2$ reseñas, una reseña es introducida en el conjunto de prueba, con $N - 1$ reseñas reservadas para entrenamiento y validación.
- En el conjunto de prueba, para cada tripla positiva (u, i, p) , que representa que el usuario u tomó la fotografía p del restaurante i , se añadieron como muestras negativas todas las triplas (u', i, p') , con todas las fotografías del mismo restaurante de otros usuarios del conjunto de entrenamiento

Todas las modificaciones producidas son exclusivamente a partir de los datos originales, no se añadieron nuevos datos externos, únicamente derivados de los datos ya existentes, sin ninguna excepción. Para cada caso de prueba, contamos con una muestra positiva: una fotografía p que no estuvo presente en los datos de entrenamiento pero que pertenece efectivamente al usuario u . El resto de fotografías de otros usuarios para el mismo restaurante actúan como muestras negativas. El objetivo de los modelos es el de clasificar las fotografías del propio usuario con mayor certeza que las de otros usuarios, teniendo una posición más alta en el ranking de imágenes creado. Esta técnica es popularmente usada en sistemas de recomendación para tareas que impliquen interacciones entre usuarios e ítems [59]. La Tabla 5.2 muestra un ejemplo de partición original de cada uno de los conjuntos de datos utilizados en este trabajo. Es importante señalar que los datasets no contienen información explícita sobre los usuarios, como sería el caso de datos demográficos. Por esta razón, cualquier modelo debe inferir las características latentes de los usuarios para generar explicaciones personalizadas, enfrentándose así a la clásica situación de “comienzo en frío” en datos de interacción usuario-ítem. Para una descripción más detallada sobre la construcción, características y particiones de los datasets, se recomienda consultar la publicación original de ELVis [8].

5.2 Métricas

En nuestros experimentos utilizamos cinco métricas diferentes para evaluar el rendimiento de los modelos de explicabilidad visual tras las diferentes aproximaciones, estas métricas fueron: AUC-ROC, Sensibilidad@k, y NDCG@k, utilizadas por trabajos previos y relevantes

Ciudad	Usuarios	Restaurantes	Fotografías	Fotos/Usuario
Gijón	5.139	598	18.679	3,64
Barcelona	33.537	5.881	150.416	4,49
Madrid	43.628	6.810	203.905	4,67

Tabla 5.1: Resumen de los datasets utilizados en términos de usuarios, restaurantes, fotografías y promedio de fotografías por usuario.

Ciudad	Usuarios	Restaurantes	Fotografías
Entrenamiento			
Gijón	5.139	598	15.190
Barcelona	33.537	5.881	120.221
Madrid	43.268	6.810	162.487
Validación			
Gijón	428	205	1.112
Barcelona	4.363	2.033	10.453
Madrid	5.852	2.301	14.276
Prueba			
Gijón	1.023	346	2.337
Barcelona	8.697	3.211	19.742
Madrid	11.874	3.643	27.142

Tabla 5.2: Partición de los datasets en entrenamiento, validación y prueba.

en tareas de ránking; además de BLEU y CHRF, dos métricas de rendimiento de sistemas de Traducción Automática con utilidad para comparar la similitud de textos.

5.2.1 Área bajo la Curva de Característica Operativa del Receptor (AUC-ROC)

El AUC-ROC mide la capacidad del modelo para distinguir entre clases positivas y negativas. Específicamente, AUC-ROC evalúa el área bajo la curva *Receiver Operating Characteristic* (ROC), que traza la Tasa de Verdaderos Positivos frente a la Tasa de Falsos Positivos. Su valor varía entre 0 y 1, donde un valor de 1 indica un desempeño perfecto, mientras que un valor de 0,5 representa un modelo que no tiene poder discriminativo. Esta métrica nos permite conocer

un comportamiento global del modelo, teniendo en cuenta los desbalances presentes en los conjuntos de datos utilizados.

Matemáticamente, se puede calcular como:

$$\text{AUC} = \frac{1}{|P||N|} \sum_{p \in P} \sum_{n \in N} \mathbb{I}(s_p > s_n),$$

donde P y N son los conjuntos de muestras positivas y negativas, s_p y s_n son los valores asignados por el modelo a las muestras positivas y negativas respectivamente, y $\mathbb{I}$ es la función indicadora.

5.2.2 Sensibilidad

El Sensibilidad@k, mide la proporción de elementos relevantes recuperados dentro de las primeras k recomendaciones realizadas por el sistema. Es una métrica común en sistemas de recomendación para evaluar qué tan bien el modelo recupera ítems relevantes para el usuario, por lo que se aplica perfectamente a la tarea de ránking de imágenes de los modelos utilizados.

Se define como:

$$\text{Sensibilidad@k} = \frac{|I_u \cap R_u(k)|}{|I_u|},$$

donde I_u es el conjunto de ítems relevantes para el usuario u , y $R_u(k)$ es el conjunto de los k ítems recomendados por el sistema. Un valor próximo a 1 de Sensibilidad@k indica que el modelo logra capturar una gran parte de los ítems relevantes dentro de las primeras k recomendaciones.

5.2.3 Normalized Discounted Cumulative Gain (NDCG)

Con la métrica **NDCG@k** se evalúa tanto la relevancia como el orden de los ítems recomendados dentro de las primeras k posiciones. Asigna un mayor peso a los ítems relevantes ubicados en posiciones superiores dentro del ránking. El cálculo de **NDCG@k** implica dos pasos:

1. Discounted Cumulative Gain (DCG)@k:

$$\text{DCG@k} = \sum_{i=1}^k \frac{2^{rel_i} - 1}{\log_2(i + 1)},$$

donde rel_i es la relevancia del ítem en la posición i .

2. **NDCGG@k (Normalización):**

$$\text{NDCG@k} = \frac{\text{DCG@k}}{\text{IDCG@k}},$$

donde IDCG@k es el valor máximo de DCG@k posible, calculado con los ítems ordenados perfectamente por relevancia.

Un valor de **NDCG@k** cercano a 1 indica que los ítems relevantes están ubicados en las posiciones más altas del ranking.

5.2.4 **Bilingual Evaluation Understudy (BLEU) y character n-gram F-score (CHRF)**

BLEU [60] y **CHRF** [61] son dos métricas de Traducción Automática, evaluando la calidad de diferentes traducciones a partir de un texto o grupo de textos base. Además de su uso para Traducción Automática, estas métricas también tienen uso para calcular la similitud de textos [62, 63], debido a esto, decidimos usarlas para evaluar la similitud de las reseñas de las imágenes que devuelve el modelo como mejor posicionadas en el ranking de cada usuario. Para obtener resultados el proceso es el siguiente: Para cada prueba del conjunto de pruebas:

1. Obtener texto de la reseña de la imagen categorizada como positiva.
2. Obtener ranking de imágenes del modelo de Explicabilidad Visual para la prueba.
3. Obtener textos de las reseñas de las 10 imágenes mejor posicionadas en el ranking.
4. Calcular métrica con texto de referencia y textos del ranking.

Con estas métricas esperamos reflejar la capacidad del modelo de no únicamente posicionar las imágenes del propio usuario en mejores posiciones del ranking, si no la propia similitud de las imágenes, o en este caso, los textos de las reseñas de donde provienen las imágenes, en comparación con las reseñas del usuario, sin diferenciar entre datos negativos y positivos. Un modelo que obtenga mejores resultados con cualquiera de estas métricas es capaz de encontrar imágenes de usuarios que si bien no son necesariamente el propio usuario, tienen gustos similares.

BLEU

BLEU es una métrica ampliamente utilizada para evaluar la calidad de texto generado automáticamente, comparándolo con un conjunto de referencias. Originalmente diseñada para tareas de traducción automática, BLEU mide la coincidencia de n -gramas entre el texto generado y las referencias, ponderando tanto la precisión como la longitud del texto generado. La

métrica se define como:

$$\text{BLEU} = BP \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right),$$

donde:

- p_n es la precisión de los n -gramas hasta un máximo de N .
- w_n son los pesos asignados a cada n -grama (generalmente uniformes).
- BP (*Brevity Penalty*) es un factor de penalización que ajusta el puntaje si el texto generado es significativamente más corto que las referencias, definido como:

$$BP = \begin{cases} 1 & \text{si } c > r, \\ \exp \left(1 - \frac{r}{c} \right) & \text{si } c \leq r, \end{cases}$$

donde c es la longitud del texto generado y r la longitud de la referencia más cercana.

CHRF

CHRF es una métrica basada en el cálculo de precisión y recuperación (*sensibilidad*) de n -gramas de caracteres, diseñada para medir la similitud entre textos. A diferencia de BLEU, CHRF opera a nivel de caracteres en lugar de palabras, lo que la hace más sensible a errores menores, como faltas de ortografía o diferencias gramaticales.

El cálculo de CHRF combina la precisión y la recuperación en forma de F -score, definido como:

$$\text{CHRF} = (1 + \beta^2) \cdot \frac{\text{Precisión} \cdot \text{Sensibilidad}}{\beta^2 \cdot \text{Precisión} + \text{Sensibilidad}},$$

donde:

- Precisión es la proporción de n -gramas de caracteres generados que aparecen en las referencias.
- Sensibilidad es la proporción de n -gramas de caracteres de las referencias que aparecen en el texto generado.
- β es un parámetro que controla la importancia relativa de la cobertura (*sensibilidad*) sobre la precisión (comúnmente $\beta = 2$).

5.2.5 Métricas de sostenibilidad

Para evaluar el impacto ambiental del desarrollo computacional en este estudio, se ha utilizado la herramienta CodeCarbon, que permite estimar las emisiones de dióxido de carbono

(CO₂) y el consumo energético asociado a la ejecución de código. Las métricas clave obtenidas incluyen el tiempo de ejecución, las emisiones de CO₂ en gramos (g) y el consumo de energía en vatios-hora (Wh).

- **Tiempo de ejecución:** Representa la duración total del proceso computacional. Este valor es fundamental para comprender la eficiencia del código y su relación con el consumo energético.
- **Emisiones de CO₂e (g):** Estimación de la cantidad de CO₂e equivalente liberado a la atmósfera como resultado del consumo eléctrico del hardware utilizado. Este valor depende de factores como la eficiencia energética del sistema y la fuente de energía empleada (renovable o basada en combustibles fósiles).
- **Consumo de energía (Wh):** Cantidad de energía utilizada durante la ejecución del código, medida en vatios-hora.

Estas métricas nos permiten evaluar y comparar el impacto ambiental de las diferentes soluciones propuestas, algo crucial para nuestro trabajo.

5.3 Híper-parámetros

Dado que hipotetizábamos que las nuevas técnicas podrían alterar el ritmo de aprendizaje del modelo debido a la mayor calidad de los datos, realizamos un reajuste específico del número de épocas utilizando el conjunto de validación de Barcelona. Optamos por no modificar el resto de hiperparámetros por dos motivos principales: limitaciones temporales y la necesidad de preservar la comparabilidad directa con los resultados previos, asegurando así la coherencia metodológica con los enfoques establecidos. Para garantizar la consistencia con investigaciones anteriores, adoptamos las mismas configuraciones de hiperparámetros reportadas en los trabajos originales de cada modelo: [ELVis](#) [8], [MF-ELVis](#) [9], y [BRIE](#) [10]. Una comparación detallada de estas configuraciones se presenta en la Tabla 5.3, además de el nuevo valor para el híper-parámetro de número de épocas de entrenamiento que conlleva el uso de las técnicas planteadas.

5.3.1 Métrica de similitud entre imágenes

Para las técnicas de selección de nuevos positivos y negativos mediante [PU Learning](#) es necesaria la elección de una métrica de similitud entre imágenes, en concreto, esta distancia es la distancia entre los *embeddings* de las imágenes. En nuestros experimentos, probamos tres métricas diferentes:

Modelo	Learning Rate	Dropout	d	épocas
ELVis	0,0005	0,2	256	100
+ PU Learning				15
+ Aumentación transformación				15
+ Aumentación genAI				50
MF-ELVis	0,0005	0,2	1024	15
+ PU Learning				5
+ Aumentación genAI				5
BRIE	0,001	0,75	64	15

Tabla 5.3: Híper-parámetros de los modelos **ELVis**, **MF-ELVis**, y **BRIE**, junto con las técnicas aplicadas y la reducción del híper-parámetro de número de épocas correspondiente a cada técnica.

- **Distancia euclídea:** Mide la distancia entre dos vectores en un espacio euclidiano y se define como:

$$d_{\text{euc}}(V_p, V_{p'}) = \sqrt{\sum_{i=1}^d (V_p[i] - V_{p'}[i])^2}$$

donde d es la dimensión del vector. Valores más bajos indican mayor similitud entre las imágenes.

- **Similitud coseno:** Mide la similitud entre dos vectores en función del ángulo entre ellos, y se define como:

$$S_{\text{cos}}(V_p, V_{p'}) = \frac{\sum_{i=1}^d V_p[i] \cdot V_{p'}[i]}{\sqrt{\sum_{i=1}^d (V_p[i])^2} \cdot \sqrt{\sum_{i=1}^d (V_{p'}[i])^2}}$$

donde $S_{\text{cos}}(V_p, V_{p'}) = 1$ indica máxima similitud y valores cercanos a -1 indican disimilitud.

- **Discrepancia Máxima de la Media (MMD) [64]:** Mide la diferencia entre distribuciones de probabilidad en espacios de características mediante funciones de kernel, y se define como:

$$\text{MMD}^2(V_p, V_{p'}) = \mathbb{E}_{V_p, V_p' \sim P}[k(V_p, V_p')] + \mathbb{E}_{V_{p'}, V_{p'}' \sim Q}[k(V_{p'}, V_{p'}')] - 2\mathbb{E}_{V_p \sim P, V_{p'}' \sim Q}[k(V_p, V_{p'}')]$$

donde $k(V_p, V_{p'})$ es una función de kernel (por ejemplo, RBF o polinomial). Valores más bajos de MMD indican mayor similitud entre distribuciones de *embeddings*.

De estas tres métricas, la similitud coseno obtiene mejores resultados que la distancia

euclídea para todos los casos probados, y resultados similares a MMD, pero con un coste computacional significativamente menor. Por esto, los resultados de selección de nuevos positivos y negativos mediante [PU Learning](#) utilizan la distancia coseno para medir la similitud entre imágenes de usuarios.

Capítulo 6

Resultados

EN este capítulo mostraremos resultados obtenidos con los diferentes métodos y técnicas detallados en el Capítulo 4, haciendo un análisis de los efectos en el rendimiento de los modelos de Explicabilidad Visual, así como el impacto en cuanto a sostenibilidad que representan.

6.1 Rendimiento global de los modelos

Los resultados obtenidos con las diferentes técnicas y métodos presentados en el Capítulo 4 utilizando los conjuntos de datos presentados en la Sección 5.1 se pueden ver en las Tablas 6.1, 6.2, y 6.3. Antes de hablar de las técnicas, lo primero, igual que en trabajos anteriores [10], BRIE obtiene los mejores resultados de manera global y para todas las métricas utilizadas, seguido de ELVis y finalmente MF-ELVis. Además de esto, se puede observar claramente una mejoría muy significativa en todas las métricas utilizadas, para todos los modelos de explicabilidad y en todos los conjuntos, con el uso de los *embeddings* producidos por el modelo ViT Large 14, esta mejoría es muy similar en porcentaje entre los diferentes conjuntos de datos y modelos utilizados, la media de las mejorías en valor relativo es la siguiente:

- AUC: 16%
- Sensibilidad@10: 30%
- NDCG@10: 46%
- BLEU: 14%
- CHRF: 3%

El conjunto de las técnicas junto con los nuevos *embeddings* de imágenes aporta una mejora de rendimiento adicional, de un 3% al AUC y 5% al Sensibilidad@10 a los modelos de ELVis

y MF-ELVis, lo que en muchos casos consigue que ELVis se acoque o iguale en rendimiento al modelo BRIE, además de permitir reducir significativamente el número de épocas de entrenamiento, lo que mejora significativamente la sostenibilidad de los modelos.

En el caso de BRIE, ninguna de las técnicas mejora el rendimiento de forma notable. Sin embargo, tras aplicar la técnica de selección de negativos mediante PU Learning, se observó una mejora estadísticamente significativa con un test de Nemenyi con $p < 0.05$. Este test de significancia estadística fue realizado ejecutando 10 veces cada versión del modelo, con y sin selección de nuevos negativos con PU Learning, creando diferentes particiones del conjunto de datos en cada ejecución para obtener resultados generalizables. Aunque la diferencia fue pequeña, demuestra la utilidad de esta técnica y justifica su estudio en trabajos futuros

6.2 Impacto en el consumo energético y emisiones

Más allá del aumento en rendimiento, en la Tabla 6.4 se evidencia que la reducción en el tamaño de los vectores de características de las imágenes (768 dimensiones en ViT Large 14 frente a 1536 en InceptionResnetV2) disminuye notablemente los tiempos de entrenamiento, el consumo energético y las emisiones de los modelos evaluados. Esta reducción oscila entre un 20% y un 25% en todas las métricas de sostenibilidad.

Si bien la fase de entrenamiento es la más intensiva en términos de consumo energético, la etapa de inferencia —es decir, la predicción o clasificación sobre nuevos datos— se convierte en el factor determinante del consumo total de energía cuando un modelo se despliega en producción a gran escala. Esta afirmación se sostiene al analizar el volumen operativo de plataformas como Tripadvisor: según datos públicos [65], la plataforma procesa más de 463 millones de visitantes mensuales y alberga 1.200 millones de reseñas y opiniones, muchas de las cuales incluyen imágenes. Si un modelo de explicabilidad visual se integrara para explicar, por ejemplo, 10 reseñas por usuario diariamente, se generarían aproximadamente 4.600 millones de inferencias mensuales (150 millones diarias). Aunque el costo energético por inferencia es menor que el de entrenamiento, el efecto acumulativo de miles de millones de operaciones mensuales superaría ampliamente el consumo puntual de la fase de entrenamiento. Esto justifica la importancia crítica de optimizar la eficiencia en inferencia para reducir la huella de carbono global de sistemas desplegados masivamente. En la Tabla 6.5 se observa que el cambio de modelo de *embeddings* reduce también el consumo energético y las emisiones de los modelos en inferencia, aproximadamente en un 15%, asegurando la sostenibilidad y el rendimiento de los modelos de explicabilidad.

Los mejores resultados de sostenibilidad con el uso de las técnicas presentadas junto con los *embeddings* producidos con ViT Large 14 se pueden ver en las Tablas 6.7, 6.8, y 6.9. Debido a que la combinación de las técnicas de mejora de la calidad de los datos y los nuevos *embed-*

dings permiten reducir significativamente el número de épocas de entrenamiento, se aprecian mejoras significativas en cuanto a sostenibilidad. En particular, se logra reducir hasta un 66% el tiempo de entrenamiento y un 40% las emisiones de CO₂ en **ELVis**, mientras que en **MF-ELVis** la reducción alcanza un 60% tanto en tiempo de entrenamiento como en emisiones. Estos resultados favorables en todos los conjuntos probados indican que las técnicas propuestas tienen un gran potencial, tanto con los conjuntos de datos utilizados como con nuevos y distintos conjuntos. Sin embargo todas las técnicas probadas permiten una reducción similar en el número de épocas de entrenamiento, únicamente se muestran las que permiten reducir este número sin afectar al rendimiento. Debido a restricciones de tiempo, no se ajustaron otros hiper-parámetros, aunque el número de épocas de entrenamiento es el que más impacta en la sostenibilidad de los modelos.

6.3 Consumo energético de las técnicas propuestas

Finalmente, en la Tabla 6.6 se presenta el consumo adicional de las diferentes técnicas utilizadas. Estas técnicas actúan como un preprocesado de los datos de entrada a los modelos, por lo que, como se mencionó previamente, su impacto es menor en comparación con la fase de inferencia, que se ejecuta con una frecuencia significativamente mayor que el preprocesado de los datos y el entrenamiento de los modelos.

6.4 Consideraciones finales sobre el costo de las técnicas

Considerando el costo de aplicar las diferentes técnicas y las mejoras que aportan en términos de sostenibilidad (ver Tabla 6.6), la Aumentación de Datos mediante Inteligencia Artificial Generativa, aunque muestra resultados prometedores, conlleva un sobre coste importante en consumo y emisiones, y su uso debería ser discutido dependiendo de las condiciones de la tarea presente. En cambio, la alternativa de selección de nuevos negativos mediante **PU Learning** ofrece resultados similares con un sobre coste significativamente menor.

Gijón					
Modelo	AUC	Sensibilidad@10	NDCG@10	BLEU	CHRF
Normal					
ELVis	0,612	0,396	0,196	6,493	27,728
MF-ELVis	0,592	0,301	0,133	6,329	27,625
BRIE	0,630	0,424	0,217	6,767	27,969
ELVis + ViT Large 14	0,702	0,492	0,262	7,341	28,474
MF-ELVis + ViT Large 14	0,691	0,486	0,278	7,297	28,369
BRIE + ViT Large 14	0,736	0,535	0,306	7,693	28,746
PU Learning para Negativos					
ELVis + ViT Large 14	0,715	0,517	0,280	7,589	28,607
MF-ELVis + ViT Large 14	0,692	0,490	0,276	7,331	28,368
BRIE + ViT Large 14	0,731	0,529	0,298	7,692	28,742
PU Learning para Positivos					
ELVis + ViT Large 14	0,720	0,531	0,292	7,634	28,711
MF-ELVis + ViT Large 14	0,703	0,498	0,279	7,384	28,426
BRIE + ViT Large 14	0,723	0,528	0,299	7,688	28,739
Aumentación de Datos por transformación de imágenes					
ELVis + ViT Large 14	0,720	0,532	0,290	7,567	28,510
MF-ELVis + ViT Large 14	0,699	0,495	0,280	7,311	28,419
BRIE + ViT Large 14	0,728	0,532	0,301	7,751	28,811
Aumentación de Datos por IA Generativa					
ELVis + ViT Large 14	0,722	0,527	0,292	7,558	28,516
MF-ELVis + ViT Large 14	0,705	0,492	0,284	7,309	28,408
BRIE + ViT Large 14	0,737	0,535	0,306	7,741	28,804

Tabla 6.1: Resultados obtenidos con el conjunto de datos de Gijón.

Barcelona					
Modelo	AUC	Sensibilidad@10	NDCG@10	BLEU	CHRF
Normal					
ELVis	0,621	0,439	0,224	7,738	29,176
MF-ELVis	0,587	0,413	0,213	7,502	29,024
BRIE	0,655	0,478	0,256	8,081	29,424
ELVis + ViT Large 14	0,726	0,562	0,320	8,804	30,050
MF-ELVis + ViT Large 14	0,696	0,527	0,304	8,493	29,761
BRIE + ViT Large 14	0,745	0,584	0,342	8,990	30,155
PU Learning para Negativos					
ELVis + ViT Large 14	0,747	0,587	0,334	8,824	30,027
MF-ELVis + ViT Large 14	0,702	0,558	0,315	8,601	29,863
BRIE + ViT Large 14	0,746	0,586	0,342	9,004	30,164
PU Learning para Positivos					
ELVis + ViT Large 14	0,726	0,558	0,313	8,804	30,050
MF-ELVis + ViT Large 14	0,699	0,531	0,308	8,511	29,854
BRIE + ViT Large 14	0,738	0,576	0,331	9,028	30,144
Aumentación de Datos por transformación de imágenes					
ELVis + ViT Large 14	0,745	0,587	0,336	8,882	30,128
MF-ELVis + ViT Large 14	6,889	0,527	0,302	8,500	29,761
BRIE + ViT Large 14	0,745	0,584	0,344	8,992	30,156
Aumentación de Datos por IA Generativa					
ELVis + ViT Large 14	0,750	0,582	0,344	8,874	30,102
MF-ELVis + ViT Large 14	0,693	0,526	0,302	8,491	29,788
BRIE + ViT Large 14	0,745	0,583	0,340	8,980	30,117

Tabla 6.2: Resultados obtenidos con el conjunto de datos de Barcelona.

Madrid					
Modelo	AUC	Sensibilidad@10	NDCG@10	BLEU	CHRF
Normal					
ELVis	0,629	0,400	0,205	7,068	28,98
MF-ELVis	0,596	0,376	0,196	6,848	28,798
BRIE	0,664	0,442	0,242	7,424	29,213
ELVis + ViT Large 14	0,737	0,522	0,298	8,161	29,788
MF-ELVis + ViT Large 14	0,699	0,490	0,281	7,899	29,592
BRIE + ViT Large 14	0,752	0,538	0,316	8,293	29,836
PU Learning para Negativos					
ELVis + ViT Large 14	0,752	0,524	0,307	8,271	29,872
MF-ELVis + ViT Large 14	0,716	0,499	0,295	8,045	29,625
BRIE + ViT Large 14	0,753	0,538	0,314	8,301	29,877
PU Learning para Positivos					
ELVis + ViT Large 14	0,729	0,511	0,298	8,166	29,790
MF-ELVis + ViT Large 14	0,700	0,480	0,276	7,837	29,592
BRIE + ViT Large 14	0,755	0,542	0,316	8,299	29,843
Aumentación de Datos por transformación de imágenes					
ELVis + ViT Large 14	0,752	0,531	0,315	8,204	29,813
MF-ELVis + ViT Large 14	0,709	0,500	0,287	7,963	29,681
BRIE + ViT Large 14	0,748	0,535	0,312	8,270	29,850
Aumentación de Datos por IA Generativa					
ELVis + ViT Large 14	0,755	0,534	0,309	8,211	29,820
MF-ELVis + ViT Large 14	0,713	0,512	0,298	8,002	29,672
BRIE + ViT Large 14	0,752	0,540	0,315	8,280	29,830

Tabla 6.3: Resultados obtenidos con el conjunto de datos de Madrid.

Modelo	Tiempo	Emisiones (g de CO2)	Consumo (Wh)
ELVis	79'12"	10,019g	57,565Wh
MF-ELVis	16'52"	3,429g	19,698Wh
BRIE	16'42"	1,985g	11,406Wh
ELVis + ViT Large 14	62'11"	7,965g	45,760Wh
MF-ELVis + ViT Large 14	13'24"	2,674g	15,316Wh
BRIE + ViT Large 14	12'33"	1,491g	8,568Wh

Tabla 6.4: Resultados de tiempo, emisiones, y consumo, para la etapa de entrenamiento del conjunto de datos de Barcelona en todos los modelos de explicabilidad visual utilizados.

Modelo	Tiempo	Emisiones (g CO2)	Consumo (Wh)
ELVis	2'25"	0,304g	1,671Wh
MF-ELVis	2'14"	0,275	1,578Wh
BRIE	2'14"	0,247	1,417Wh
ELVis + ViT Large 14	2'01"	0,258g	1,423Wh
MF-ELVis + ViT Large 14	1'56"	0,229g	1,314Wh
BRIE + ViT Large 14	1'58"	0,215g	1,214Wh

Tabla 6.5: Resultados de tiempo, emisiones, y consumo, para 10 ejecuciones de la inferencia del conjunto de prueba, para el conjunto de datos de Barcelona en todos los modelos de explicabilidad visual utilizados.

Técnica	Tiempo	Emisiones (g de CO2)	Consumo (Wh)
<i>Embeddings</i> con ViT Large 14	354'32"	76,976g	442,248Wh
Aumentación por genAI	440'10"	91,325g	526,453Wh
Aumentación por transformación de imágenes	16'52"	3,429g	19,698Wh
PU Negativos	2'48"	0,241g	1,385Wh
PU Positivos (preprocesado)	12'02"	0,976g	5,607Wh

Tabla 6.6: Resultados de tiempo, emisiones, y consumo, para las técnicas presentadas utilizando los *embeddings* producidos por el modelo ViT Large 14 para el conjunto de datos de Barcelona, Aumentación por genAI incluye la generación de imágenes y su posterior transformación a *embeddings* y adición al conjunto de datos.

Gijón				
Modelo	Épocas	Tiempo	Consumo	Emisiones
PU Learning para Negativos				
ELVis + ViT Large 14	15	1'44"	0,212g	1,215Wh
MF-ELVis + ViT Large 14	5	0'48"	0,103g	0,591Wh
Aumentación de Datos por transformación de imágenes				
ELVis + ViT Large 14	15	1'41"	0,204g	1,175Wh
Aumentación de Datos por IA Generativa				
ELVis + ViT Large 14	50	5'50"	0,708g	4,058Wh
MF-ELVis + ViT Large 14	5	0'52"	0,112g	0,624Wh

Tabla 6.7: Resultados de tiempo de entrenamiento ,consumo para las mejores aproximaciones tras la optimización del hiper-parámetro de épocas de entrenamiento, para el conjunto de datos de Gijón y con los *embeddings* producidos con ViT Large 14.

Barcelona				
Modelo	Épocas	Tiempo	Emisiones (g de CO2)	Consumo (Wh)
PU Learning para Negativos				
ELVis + ViT Large 14	15	20'05"	4,624g	26,567Wh
MF-ELVis + ViT Large 14	5	5'44"	0,935g	6,178Wh
Aumentación de Datos por transformación de imágenes				
ELVis + ViT Large 14	15	19'34"	4,425g	26,256Wh
Aumentación de Datos por IA Generativa				
ELVis + ViT Large 14	50	62'42"	11'37"	69,935Wh
MF-ELVis + ViT Large 14	5	5'34"	0,912g	6,110Wh

Tabla 6.8: Resultados de tiempo de entrenamiento ,consumo para las mejores aproximaciones tras la optimización del hiper-parámetro de épocas de entrenamiento, para el conjunto de datos de Barcelona y con los *embeddings* producidos con ViT Large 14.

Madrid				
Modelo	Épocas	Tiempo	Consumo	Emisiones
PU Learning para Negativos				
ELVis + ViT Large 14	15	23'42"	5,318g	30,930Wh
MF-ELVis + ViT Large 14	5	7'53"	1,539g	8,843Wh
Aumentación de Datos por transformación de imágenes				
ELVis + ViT Large 14	15	22'08"	5,093g	28,129Wh
Aumentación de Datos por IA Generativa				
ELVis + ViT Large 14	50	73'21"	18,205g	81,554Wh
MF-ELVis + ViT Large 14	5	8'12"	1,623	9,014Wh

Tabla 6.9: Resultados de tiempo de entrenamiento ,consumo para las mejores aproximaciones tras la optimización del hiper-parámetro de épocas de entrenamiento, para el conjunto de datos de Madrid y con los *embeddings* producidos con ViT Large 14.

Entregable: Librería para mejora de sistemas de explicabilidad visual

COMO entregable de este proyecto, hemos producido una librería con las soluciones propuestas, de código abierto y público en Github ¹. Las técnicas incluídas en la librería tienen como nexo común el mejorar los datos de entrada para modelos de explicabilidad visual, lo que separa la propia librería de los modelos utilizados en nuestros experimentos, pudiéndose utilizar las técnicas incluídas en modelos diferentes, aunque con la misma tarea de aprender un ranking de imágenes por usuarios. Más allá de los resultados mostrados en el Capítulo 6, se incluyen todas las técnicas propuestas, ya que son aproximaciones de valor para la comunidad, con el potencial de ser de gran utilidad para nuevos conjuntos de datos y/o diferentes modelos. La librería cuenta con un archivo *requirements.txt* donde se especifican todas las dependencias necesarias para su correcto funcionamiento, facilitando su instalación y uso. Además, su diseño modular permite que las funciones sean utilizadas de manera independiente o integradas dentro de flujos de trabajo más complejos. Además del repositorio en Github, la librería se ha cargado a la plataforma PyPi ², lo que permite una simple instalación de la librería en la versión deseada, puede descargarse de la siguiente manera:

```
1 pip install data-improvement-library
```

La librería tiene las siguientes dependencias:

- `numpy==1.26.4`
- `pandas==2.2.3`
- `Pillow==9.4.0`

¹ El repositorio de Github se encuentra en: https://github.com/david-esteban-martinez/data_improvement_library

² La librería en PyPi se encuentra en <https://pypi.org/project/data-improvement-library/>

- `pytorch_lightning==2.4.0`
- `timm==1.0.14`
- `torch==2.4.1`
- `torchvision==0.19.1`
- `tqdm==4.65.0`

7.1 Buenas Prácticas en el Desarrollo de la Librería

Para garantizar la calidad, extensibilidad y eficiencia del código, se han seguido una serie de buenas prácticas en el desarrollo de la librería. Estas incluyen la adherencia a estándares de codificación, modularidad, gestión de dependencias y manejo eficiente de datos.

7.1.1 Cumplimiento de Estándares de Código

El código sigue las convenciones establecidas en la guía de estilo PEP8 [66] para Python. Algunas de las principales consideraciones incluyen:

- Uso de nombres de variables y funciones descriptivos, siguiendo la convención `snake_case`.
- Separación clara de responsabilidades en funciones bien definidas.
- Inclusión de docstrings en formato estándar para documentar cada función, como se muestra en el siguiente ejemplo:

```
1 def X_transform(img, apply_all=False):
2     """
3     Aplica una transformación aleatoria a la imagen dada.
4
5     Args:
6         img (PIL.Image): Imagen a transformar.
7         apply_all (bool): Si es True, aplica todas las
8         transformaciones y devuelve una lista.
9
10    Returns:
11        Imagen(es) transformada(s) como PIL.Image.
12    """
```

Listing 7.1: Ejemplo de docstring en una función de transformación de imágenes.

Estas prácticas son necesarias para facilitar la legibilidad y mantenimiento del código, ya que el objetivo principal es que pueda ser utilizado por profesionales de diversos perfiles.

7.1.2 Modularidad y Reutilización del Código

El diseño modular de la librería permite que cada funcionalidad esté separada en archivos independientes. Esto mejora la reutilización y facilita la incorporación de nuevas funcionalidades sin afectar el código existente.

Por ejemplo, la funcionalidad de extracción de embeddings se encuentra en el módulo `new_embeddings.py`, y ambas técnicas de selección de nuevas muestras con [PU Learning](#) se separan en diferentes módulos `pu_positives.py` para la selección de nuevos positivos, y `pu_negatives.py` para la selección de nuevos negativos.

Además, la modularidad de la librería permite una desacomplación completa del módulo de explicabilidad visual para el que se quieran mejorar los datos de entrenamiento, separando los datos del modelo o modelos utilizados, e incluso a nivel de entorno, pudiéndose utilizar Pytorch para la librería y otras herramientas como Tensorflow para la implementación de los modelos de explicabilidad visual. La única excepción es la funcionalidad como *callback* del módulo de selección de positivos mediante [PU Learning](#), ya que solo puede utilizarse para modelos implementados con Pytorch con configuración específica para la tarea.

7.2 Diseño y Arquitectura de la Librería

7.2.1 Estructura General

La librería desarrollada sigue un enfoque modular para facilitar la integración y extensión de sus funcionalidades. Se compone de cuatro módulos principales:

- **Módulo de Aumentación de Datos** (`data_augmentation.py`): Encargado de aplicar transformaciones a imágenes y de generar nuevos conjuntos de datos a partir de imágenes generadas con Inteligencia Artificial. Corresponde a las técnicas de **aumentación de los datos e Inteligencia Artificial Generativa**, en Sección [4.2.1](#) y Sección [4.2.2](#) respectivamente.
- **Módulo de Generación de Embeddings** (`new_embeddings.py`): Responsable de extraer representaciones numéricas (embeddings) de imágenes en el formato correcto, utilizando modelos de [AA](#). Se corresponde a la técnica de **mejora de embeddings de imágenes** presentada en la Sección [4.3](#)
- **Módulo de Selección de Negativos con [PU Learning](#)** (`pu_negatives.py`): Implementa la selección de nuevos negativos mediante similitud de las imágenes de otros usuarios, corresponde a la técnica de **selección de nuevos negativos mediante similitud de imágenes**, presentada en la Sección [4.1.1](#)

- **Módulo de Selección de Positivos con PU Learning** (`pu_positives.py`): Implementa la selección de nuevos positivos mediante similitud de las imágenes de otros usuarios, corresponde a la técnica de **selección de nuevos positivos mediante similitud de usuarios**, presentada en la Sección 4.1.2

Cada módulo se ha diseñado con el propósito de ser reutilizable y escalable, permitiendo la fácil integración de nuevas técnicas en el futuro.

7.2.2 Módulo de aumentación de Datos

El módulo de aumentación de datos (`data_augmentation.py`) aplica diversas transformaciones a imágenes con el fin de enriquecer el conjunto de datos de entrenamiento. Corresponde a la técnica presentada en la sección 4.2.1. Tiene otra funcionalidad, que es la de generar nuevos *embeddings* de imágenes sin aplicar transformaciones, como hicimos con las imágenes generadas con Inteligencia Artificial Generativa, y añadirlas como nuevos positivos a su usuarios correspondiente en el conjunto de datos.

La función principal, `transform_image()`, carga imágenes, aplica las transformaciones o procesa las imágenes generadas, y genera una nueva versión del conjunto de datos, almacenando tanto las imágenes transformadas como sus metadatos.

7.2.3 Módulo de Generación de Embeddings

El módulo `new_embeddings.py` corresponde a la técnica presentada en la Sección 4.3, se encarga de extraer *embeddings* de imágenes utilizando modelos preentrenados de **Aprendizaje Automático**. El modelo por defecto es ViT Large 14, de `timm`, una biblioteca que proporciona modelos de clasificación de imágenes de alto rendimiento, aunque permite la utilización de un modelo personalizado.

El proceso sigue los siguientes pasos:

1. Carga de imágenes desde el directorio especificado.
2. Extracción de *embeddings* a partir de un modelo **AA**, por defecto ViT Large 14.
3. Almacenamiento de los *embeddings* en formato `pickle` para su posterior en los modelos de explicabilidad visual.

7.2.4 Módulo de Selección de Negativos con PU Learning

El módulo `pu_negatives.py`, corresponde a la técnica presentada en la Sección 4.1.1, implementa una estrategia basada en similitud de *embeddings* de imágenes para la selección de muestras negativas.

Esta selección de negativos sigue el siguiente proceso:

- **Carga de Datos:** Se cargan los datos en formato `pickle`.
- **Cálculo de Centroides de Usuario:** Se determina un vector promedio representativo de las imágenes de cada usuario.
- **Selección de Negativos:** Se seleccionan como negativos muestras de otros usuarios, tanto de restaurantes diferentes como del mismo restaurante, asegurando que las imágenes seleccionadas sean suficientemente distintas de las imágenes preferidas por el usuario mediante la distancia al centroide del usuario.

7.2.5 Módulo de Selección de Positivos con **PU Learning**

El módulo `pu_positives.py` implementa una estrategia basada en centroides de usuario para la selección de muestras positivas. Corresponde a la técnica presentada en la Sección 4.1.2

Las funciones clave de este módulo incluyen:

- **Cálculo de Centroides de Usuario:** Se obtiene un vector promedio para cada usuario.
- **Remuestreo de Positivos:** Se seleccionan nuevas muestras positivas a partir de los vecinos más cercanos al centroide de cada usuario.
- **Callback para Entrenamiento:** Se implementa una clase personalizada (`CallbackEnd`) que actualiza los centroides y remuestrea positivos en cada época del entrenamiento.

El objetivo principal del módulo es ser utilizado como *callback* o retrollamada al acabar cada época de entrenamiento, con el objetivo de añadir gradualmente nueva información en entrenamiento y evitar el sobre-entrenamiento.

7.3 Guía de Uso

Cada módulo de la librería requiere una serie de variables de entrada que definen su funcionamiento. En esta sección, se describen estas variables y se proporciona un ejemplo de uso para cada módulo.

7.3.1 Módulo de Aumentación de Datos

Variables de Entrada

- `data_dir` (str): Ruta al archivo de datos de entrada con las interacciones de los usuarios.

- `vector_dir` (str): Ruta al archivo que contiene los embeddings de las imágenes.
- `image_dir` (str): Ruta al directorio donde se encuentran almacenadas las imágenes.
- `output_dir` (str): Ruta donde se almacenarán los datos procesados.
- `output_name` (str, opcional): Nombre del archivo de salida. Por defecto, `TRAIN_IMG`.
- `embedding_model` (`torch.nn.Module`, opcional): Modelo preentrenado para la extracción de embeddings. De dejarse vacío, se utiliza por defecto un ViT Large 14 preentrenado.
- `no_aug` (bool, opcional): Si es `True`, desactiva la aplicación de aumentación de datos.
- `batch_size` (int, opcional): Número de imágenes procesadas por lote. Valor por defecto: 32.
- `apply_all` (bool, opcional): Si es `True`, aplica todas las transformaciones disponibles.
- `labels` (list, opcional): Lista con los nombres de las columnas en el dataset.

Variables de Salida

No produce ninguna salida, el propio módulo se encarga de guardar los datos en formato `pickle` a partir de las variables de entrada.

Ejemplo de Uso

```
1 from data_augmentation import augment_data
2
3 # Procesar imágenes con aumentación de datos y extracción de
  embeddings
4 augment_data(data_dir="data/user_data.pkl",
5             vector_dir="data/image_vectors.pkl",
6             image_dir="images/", output_dir="processed_data/",
7             output_name="TRAIN_IMG", embedding_model=None,
8             no_aug=False, batch_size=32, apply_all=True, labels=None)
```

Listing 7.2: Ejemplo de código para el módulo de aumentación de datos.

7.3.2 Módulo de Generación de *Embeddings* (`new_embeddings.py`)

Variables de Entrada

- `directory` (str): Directorio donde se encuentran las imágenes.
- `output_dir` (str): Directorio donde se almacenarán los embeddings generados.
- `output_name` (str): Nombre del archivo de salida.
- `embedding_model` (`torch.nn.Module` o `None`): Modelo preentrenado para la extracción de embeddings. De dejarse vacío, se utiliza por defecto un ViT Large 14 preentrenado,
- `batch_size` (int): Número de imágenes procesadas por lote.

Variables de Salida

No produce ninguna salida, el propio módulo se encarga de guardar los datos en formato `pickle` a partir de las variables de entrada.

Ejemplo de Uso

```

1 from new_embeddings import create_new_embeddings
2
3 # Generar embeddings para un conjunto de imágenes
4 create_new_embeddings(directory="images/",
5                       output_dir="embeddings/", output_name="img_vectors.pkl",
6                       embedding_model=None, batch_size=32)

```

Listing 7.3: Ejemplo de código para el módulo de generación de *embeddings*.

7.3.3 Módulo de Selección de Negativos con *PU Learning* (`pu_negatives.py`)

Variables de Entrada

- `data_file` (str): Ruta al archivo de datos de interacción de usuarios.
- `vector_file` (str): Ruta al archivo de *embeddings* de imágenes.
- `outdir_name` (str): Directorio de salida donde se guardará el dataset procesado.
- `centroid` (int, opcional): Percentil del centroide (por defecto 90).
- `factor` (float, opcional): Factor de ajuste de distancias.
- `labels` (list, opcional): Etiquetas de las columnas en el conjunto de datos.

Variables de Salida

No produce ninguna salida, el propio módulo se encarga de guardar los datos en formato `pickle` a partir de las variables de entrada.

Ejemplo de Uso

```
1 from pu_negatives import resample_negatives
2
3 # Generar muestras negativas equilibradas
4 resample_negatives(data_file="data/user_data.pkl",
5                   vector_file="data/image_vectors.pkl",
6                   outdir_name="processed_data/", centroid=90, factor=1.0)
```

Listing 7.4: Ejemplo de código para el módulo de selección de nuevos negativos.

7.3.4 Módulo de Selección de Positivos con **PU Learning** (`pu_positives.py`)

El módulo se puede utilizar de dos formas principales:

- Como **callback** en el entrenamiento de modelos con `PyTorch Lightning`, permitiendo actualizar los centroides de usuario dinámicamente en cada época.
- Como **preprocesado**, aplicándose antes del entrenamiento al conjunto de datos.

Uso como *Callback*

El módulo proporciona la clase `CallbackEnd`, que extiende la funcionalidad de `Callback` de `PyTorch Lightning`. Su propósito es actualizar los centroides de usuario y realizar un remuestreo de las muestras positivas al final de cada época de entrenamiento.

Para su correcto funcionamiento sin adaptaciones, requiere que el modelo tenga ciertas características:

- **Dataset en `train_data_loader`** debe contener:
 - `datamodule.image_embeddings`: tensor de *embeddings* de imágenes.
 - `dataframe`: debe incluir columnas `id_user`, `id_img`, `id_restaurant`, `take`.
- **Acceso a *embeddings***: Indexable por `id_img` en el `dataframe`.
- **Formato de *embeddings***: Debe ser un tensor convertible a `NumPy` (`.cpu().detach().numpy()`).

- **Muestreo de datos:** Debe permitir modificar `pu_dataset` en el dataset.
- **Requisitos en el DataModule:** Proveer `image_embeddings` y manejar `pu_dataset` correctamente.

Ejemplo de Uso

```
1 from pu_positives import CallbackEndPositives
2 from pytorch_lightning import Trainer
3
4 # Crear instancia del callback
5 callback_end = CallbackEndPositives()
6
7 # Configurar el entrenador con el callback
8 trainer = Trainer(callbacks=[callback_end])
9
10 # Iniciar el entrenamiento
11 trainer.fit(model)
```

Listing 7.5: Implementación del callback para selección de nuevos positivos.

Uso como Preprocesado

Además de su integración como *callback*, este módulo permite generar un dataset con nuevos positivos seleccionados a través de la similaridad entre usuarios antes del entrenamiento. Esto se logra calculando centroides de usuario y realizando un remuestreo de muestras positivas. La funcionalidad está separada en dos funciones, `centroid_users`, para calcular los centroides de los usuarios, y `resample_positives`, función encargada de actualizar el conjunto de datos con nuevas muestras positivas utilizando centroides como filtro por similaridad coseno.

variables de Entrada

Para la función `centroid_users`:

- `dataframe` (`pandas.DataFrame`): Conjunto de datos de pares usuario-imagen.
- `vectors` (`numpy.array`): Representaciones en *embeddings* de las imágenes de los usuarios.
- `labels` (`list`, opcional): Lista con los nombres de las columnas en el dataset.

Para la función `resample_positives`:

- `dataframe` (`pandas.DataFrame`): Conjunto de datos de pares usuario-imagen.
- `centroids` (`dict`): Diccionario, las claves representan el ID del usuario, y los valores son los centroides de las imágenes de los usuarios, en forma de *embedding*.
- `k` (`int`): Indicador de cuantos usuarios similares son considerados para añadir las muestras positivas para cada usuario.

Variables de Salida

La función `centroid_users` devuelve `centroids`, un diccionario con los centroides de los usuarios con claves el ID del usuario y valor el centroe de las imágenes del usuario, en forma de *embedding*.

La función `resample_positives` devuelve el `dataframe` de entrada actualizado con las nuevas muestras positivas.

Ejemplo de Uso

```
1 from pu_positives import centroid_users, resample_positives
2 import pandas as pd
3 import numpy as np
4
5 # Cargar el conjunto de datos
6 dataframe = pd.read_pickle("data/user_data.pkl")
7
8 # Cargar los embeddings de las imágenes
9 image_vectors = np.load("data/image_vectors.npy")
10
11 # Calcular centroides de usuario
12 centroids = centroid_users(dataframe, image_vectors)
13
14 # Generar un nuevo dataset con muestras positivas balanceadas
15 new_dataframe = resample_positives(dataframe, centroids, 3)
16
17 # Guardar el dataset procesado
18 new_dataframe.to_pickle("processed_data/balanced_dataset.pkl")
```

Listing 7.6: Ejemplo de código para el módulo de selección de nuevos positivos.

Conclusiones

ESTE trabajo ha abordado algunos de los desafíos contemporáneos en sistemas de recomendación, centrándose en tres pilares fundamentales: personalización, explicabilidad, y sostenibilidad. Para ello, hemos seguido de manera rigurosa la metodología [CRISP-DM](#), explorando el problema y desarrollando soluciones de manera iterativa, reemplazando la fase final tradicional de despliegue de un modelo, a la creación y publicación de una librería que recopila las técnicas propuestas para mejorar el rendimiento de los modelos de explicabilidad visual sin comprometer la sostenibilidad de los mismos. Para ello, utilizamos diversas técnicas innovadoras, presentadas en el Capítulo 4: 1) Selección de nuevos positivos y negativos con Aprendizaje Positivo y Sin Etiquetas, 2) Aumentación de datos por transformación de imágenes o IA Generativa, y 3) Mejora de *embeddings* de imágenes.

8.1 Contribuciones principales

En nuestro trabajo, para resolver la problemática de la falta de transparencia en Sistemas de Recomendación, con énfasis en la sostenibilidad de las soluciones, hicimos una investigación del Estado del Arte en el campo, y llevamos a cabo un proceso de desarrollo de diferentes técnicas que buscan mejorar la calidad de los datos de entrada de modelos de explicabilidad visual. En concreto, nuestras soluciones están diseñadas para [SR](#) basados en imágenes subidas por usuarios, como es el caso de las reseñas de Tripadvisor. Nuestras contribuciones ayudan de manera directa a solucionar algunos de los problemas más importantes en soluciones anteriores a través de técnicas novedosas:

- Los datos utilizados presentan una gran dispersión debido a su naturaleza como reseñas de usuarios, con unos pocos usuarios teniendo gran número de reseñas y el resto viéndose en comparación poco representados. Proponemos como solución a este problema el uso de técnicas de aumentación de los datos de los usuarios menos representados mediante transformación de sus imágenes, o mediante la generación de nuevas imágenes

a partir de sus reseñas con IA generativa.

- La selección de negativos en el Estado del Arte actual es mediante la selección aleatoria de imágenes de diferentes usuarios, lo que puede introducir como muestras negativas imágenes con el potencial de explicar los gustos del usuario y ser muestras positivas realmente. Proponemos una nueva técnica para seleccionar las muestras negativas teniendo en cuenta la similaridad entre las imágenes de los usuarios, escogiendo imágenes menos similares como positivos fiables.
- Actualmente, para evitar un desbalance en las muestras positivas de los usuarios y las muestras negativas, se lleva a cabo una duplicación de las muestras positivas de cada usuario, lo que no añade información nueva que puedan utilizar los modelos de explicabilidad. Para solucionar esto, proponemos una técnica de selección de nuevas muestras positivas para cada usuario teniendo en cuenta la similaridad de imágenes entre otros usuarios.
- La calidad de los datos de entrada en los modelos del Estado del Arte actual es una gran barrera para la obtención de mejores resultados. Proponemos la utilización de nuevos modelos de *embedding* de imágenes de Estado del Arte con el objetivo de mejorar la calidad de los datos y extraer más información útil de las características de las imágenes de los usuarios que puedan utilizar los modelos de explicabilidad visual para tomar decisiones.

Con nuestras técnicas propuestas, hemos podido obtener resultados que sobrepasan los del Estado del Arte actual, incrementando de un 15 a 45% el rendimiento de los modelos utilizados para las diferentes métricas de rendimiento usadas. Junto con el incremento notable en rendimiento que proporcionan nuestras técnicas, también aumentan la sostenibilidad de los modelos, disminuyendo el consumo energético y las emisiones en un 25% para la etapa de entrenamiento y en un 15% para la etapa de inferencia, además la utilización de varias técnicas propuestas simultáneamente aumenta el rendimiento en un 5% adicional, y permite la reducción del hiper-parámetro de épocas de entrenamiento, lo que reduce significativamente el tiempo de entrenamiento y las emisiones, hasta un 66% en *ELVis* y un 60% en *MF-ELVis*. Finalmente, hemos incluido estas técnicas en una librería pública en Python, escrita de forma modular y separando las técnicas propuestas de los modelos de explicabilidad utilizados, permitiendo su implementación o su uso para mejorar posteriores por la comunidad. Esta librería está disponible en Github ¹ y en PyPi ², lo que junto con su exhaustiva documentación asegura facilidad a la hora de utilizarla.

¹ El repositorio de Github se encuentra en: https://github.com/david-esteban-martinez/data_improvement_library

² La librería en PyPi se encuentra en <https://pypi.org/project/data-improvement-library/>

8.2 Conclusiones del alumno

Como alumno, he podido expandir sobre los conocimientos aprendidos durante el máster, trabajando bajo una metodología profesional de desarrollo como es CRISP-DM y analizando el problema desde el punto de vista del negocio y del usuario, con el objetivo de resolver la problemática presentada en la falta de transparencia en [SR](#) de la forma mas sostenible posible. Además de esto, pude profundizar en mis conocimientos en Inteligencia Artificial y análisis de datos, trabajando con grandes volúmenes de datos de casos reales, además del énfasis en código de calidad y documentado para el uso posterior por la comunidad.

8.3 Trabajo Futuro

Considerando los avances logrados en este trabajo, aún existen desafíos pendientes y oportunidades para futuras investigaciones:

- Para las técnicas de selección de negativos y positivos con [PU Learning](#) se utilizó como métrica de similaridad de las imágenes la similaridad coseno, y se probaron como alternativas la distancia euclídea y la Discrepancia Media Máxima (MMD). Es posible que métodos más avanzados para medir la similaridad entre imágenes puedan aportar importantes beneficios a esta técnica.
- Los conjuntos de datos utilizados son de reseñas de Tripadvisor, los únicos existentes públicamente con las características necesarias para evaluar la problemática, es decir, conjuntos de Sistemas de Recomendación basados en imágenes de usuarios. La creación de nuevos conjuntos que cumplan estas características, pero de otros campos permitirían evaluar el comportamiento de las técnicas propuestas en entornos diferentes.
- Nuestro enfoque se basa en la evaluación de la retroalimentación implícita, asumiendo que la imagen propia del usuario sería la mejor para explicar la recomendación que recibe. Aún se podría explorar la retroalimentación explícita, donde se obtiene información directa del usuario sobre sus preferencias, lo que en conjunto con nuestra evaluación aportaría una visión global del problema.
- Aunque se utilizaron las técnicas de transformaciones aleatorias de imágenes con mejor rendimiento según la literatura, sería de interés una investigación profunda en este campo para integrar nuevas e innovadoras transformaciones y observar el efecto en los modelos.
- Una de las problemáticas con las que tratamos es la de “comienzo en frío”, ya que los usuarios con menor representación en el conjunto de datos reciben explicaciones de

menor calidad que aquellos con mayor volumen de imágenes, al tener un menor porcentaje de los datos de entrenamiento. Nosotros tratamos con este problema para los usuarios con menor número de imágenes, pero el problema sigue existiendo para aquellos usuarios que no tengan aún imágenes o reseñas en texto, por lo que una solución que afecte a este tipo de usuario tendría grandes beneficios.

Lista de acrónimos

- AA** Aprendizaje Automático. 1–3, 11, 13, 22, 31, 32, 59, 60
- AUC-ROC** Área bajo la Curva de Característica Operativa del Receptor. 41, II
- BERT** Bidirectional Encoder Representations from Transformers. 34
- BLEU** Bilingual Evaluation Understudy. 43, 44, II
- BPR** Bayesian Pairwise Ranking. 20, 21
- BRIE** Bayesian Ranking of Images for Explainability. 1, 10, 18, 20, 21, 28, 45, 46, 49, 50, II, VII
- CB** Content Based. 13, 15
- CF** Collaborative Filtering. 14, 15
- CHRF** character n-gram F-score. 43, 44, II
- CITIC** Centro de Investigación en TIC. 33
- CLIP** Contrastive Language-Image Pretraining. 37
- CNN** Convolutional Neural Networks. 15, 17, 18
- CRISP-DM** Cross Industry Standard Process for Data Mining. 6, 8, 67, I
- DCG** Discounted Cumulative Gain. 42
- ELVis** Explaining Likings Visually. 1, 10, 17, 18, 20–22, 28, 39, 45, 46, 49–51, 68, II, VII
- FC** Fully Connected. 18
- i.i.d.** independiente e idénticamente distribuida. 28

IA Inteligencia Artificial. 2, 13, 32

IDE Entorno de Desarrollo Integrado. 10

LIME Local Interpretable Model-agnostic Explanation. 1

MF-ELVis Matrix Factorization ELVis. 1, 10, 18, 20–22, 28, 45, 46, 49–51, 68, II, VII

MIM Masked Image Model. 34

MLP Perceptrón Multicapa. 18

NDCG Normalized Discounted Cumulative Gain. 42, 43, II

NLP Procesamiento de Lenguaje Natural. 11

PU Learning Positive Unlabelled Learning. 8, 9, 21, 23, 27, 29, 30, 45–47, 50–56, 59–61, 63, 64, 69, III, V

Retorno de Inversión ROI. 5

RNN Recurrent Neural Networks. 15

ROC Receiver Operating Characteristic. 41

SCAR Select Completely At Random. 28

SHAP SHapley Additive exPlanations. 1

SR Sistemas de Recomendación. 1, 13–16, 18, 20, 27, 67, 69, I

VBPR Visual Bayesian Personalized Ranking. 17

ViT Vision Transformer. 37, 38, 60, 62, 63, V

Bibliografía

- [1] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, “Pytorch: An imperative style, high-performance deep learning library,” no. arXiv:1912.01703, Dec. 2019, arXiv:1912.01703 [cs]. [En línea]. Disponible en: <http://arxiv.org/abs/1912.01703>
- [2] W. Falcon and T. P. L. team, “Pytorch lightning,” Mar. 2019. [En línea]. Disponible en: <https://github.com/Lightning-AI/lightning>
- [3] “pickle — python object serialization.” [En línea]. Disponible en: <https://docs.python.org/3/library/pickle.html>
- [4] T. p. d. team, “pandas-dev/pandas: Pandas,” Feb. 2020. [En línea]. Disponible en: <https://doi.org/10.5281/zenodo.3509134>
- [5] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant, “Array programming with NumPy,” *Nature*, vol. 585, no. 7825, pp. 357–362, Sep. 2020. [En línea]. Disponible en: <https://doi.org/10.1038/s41586-020-2649-2>
- [6] A. Lacoste, A. Luccioni, V. Schmidt, and T. Dandres, “Quantifying the carbon emissions of machine learning,” no. arXiv:1910.09700, Nov. 2019, arXiv:1910.09700 [cs]. [En línea]. Disponible en: <http://arxiv.org/abs/1910.09700>
- [7] Feb. 2025. [En línea]. Disponible en: <https://huggingface.co/>
- [8] J. Díez, P. Pérez-Núñez, O. Luaces, B. Remeseiro, and A. Bahamonde, “Towards explainable personalized recommendations by learning from users’ photos,” *Information Sciences*, vol. 520, p. 416–430, May 2020.

-
- [9] J. Paz-Ruza, C. Eiras-Franco, B. Guijarro-Berdiñas, and A. Alonso-Betanzos, “Sustainable personalisation and explainability in dyadic data systems,” *Procedia Computer Science*, vol. 207, p. 1017–1026, Jan. 2022.
- [10] J. Paz-Ruza, A. Alonso-Betanzos, B. Guijarro-Berdiñas, B. Cancela, and C. Eiras-Franco, “Sustainable transparency on recommender systems: Bayesian ranking of images for explainability,” *Information Fusion*, vol. 111, p. 102497, Nov. 2024.
- [11] A. Fernández-Campa-González, J. Paz-Ruza, A. Alonso-Betanzos, and B. Guijarro-Berdiñas, “Positive-unlabelled learning for improving image-based recommender system explainability,” no. arXiv:2407.06740, Jul. 2024, arXiv:2407.06740 [cs]. [En línea]. Disponible en: <http://arxiv.org/abs/2407.06740>
- [12] A. Shazia, T. Z. Xuan, J. H. Chuah, J. Usman, P. Qian, and K. W. Lai, “A comparative study of multiple neural network for detection of covid-19 on chest x-ray,” *EURASIP Journal on Advances in Signal Processing*, vol. 2021, no. 1, p. 50, Jul. 2021.
- [13] J. Maurício, I. Domingues, and J. Bernardino, “Comparing vision transformers and convolutional neural networks for image classification: A literature review,” *Applied Sciences*, vol. 13, no. 9, p. 5521, 2023.
- [14] F. Ricci, L. Rokach, and B. Shapira, *Introduction to Recommender Systems Handbook*. Boston, MA: Springer US, 2011, p. 1–35. [En línea]. Disponible en: https://doi.org/10.1007/978-0-387-85820-3_1
- [15] Y. Zhang and X. Chen, “Explainable recommendation: A survey and new perspectives,” *Foundations and Trends® in Information Retrieval*, vol. 14, no. 1, p. 1–101, 2020, arXiv:1804.11192 [cs].
- [16] N. Tintarev and J. Masthoff, *Designing and Evaluating Explanations for Recommender Systems*. Boston, MA: Springer US, 2011, p. 479–510. [En línea]. Disponible en: https://doi.org/10.1007/978-0-387-85820-3_15
- [17] Y. Lou, R. Caruana, J. Gehrke, and G. Hooker, “Accurate intelligible models with pairwise interactions,” in *Proceedings of the 19th ACM SIGKDD international conference on Knowledge discovery and data mining*. Chicago Illinois USA: ACM, Aug. 2013, p. 623–631. [En línea]. Disponible en: <https://dl.acm.org/doi/10.1145/2487575.2487579>
- [18] S. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” no. arXiv:1705.07874, Nov. 2017, arXiv:1705.07874 [cs]. [En línea]. Disponible en: <http://arxiv.org/abs/1705.07874>

- [19] M. T. Ribeiro, S. Singh, and C. Guestrin, ““why should i trust you?”: Explaining the predictions of any classifier,” no. arXiv:1602.04938, Aug. 2016, arXiv:1602.04938 [cs]. [En línea]. Disponible en: <http://arxiv.org/abs/1602.04938>
- [20] J. Zhong and E. Negre, “Shap-enhanced counterfactual explanations for recommendations,” in *Proceedings of the 37th ACM/SIGAPP Symposium on Applied Computing*, ser. SAC '22. New York, NY, USA: Association for Computing Machinery, May 2022, p. 1365–1372. [En línea]. Disponible en: <https://dl.acm.org/doi/10.1145/3477314.3507029>
- [21] Y. Wu, L. Zhang, U. A. Bhatti, and M. Huang, “Interpretable machine learning for personalized medical recommendations: A lime-based approach,” *Diagnostics*, vol. 13, no. 1616, p. 2681, Jan. 2023.
- [22] F. Yin, R. Fu, X. Feng, T. Xing, and M. Ji, “An interpretable neural network tv program recommendation based on shap,” *International Journal of Machine Learning and Cybernetics*, vol. 14, no. 10, p. 3561–3574, Oct. 2023.
- [23] N. P, M. V, D. A, B. K, A. M, and R. C, “A prediction and recommendation system for diabetes mellitus using xai-based lime explainer,” in *2022 International Conference on Sustainable Computing and Data Communication Systems (ICSCDS)*, Apr. 2022, p. 1472–1478. [En línea]. Disponible en: https://ieeexplore.ieee.org/abstract/document/9760847?casa_token=f7NE0uHiEBAAAAAA:wKtyBSDqpDsFquP02zN8K1UIK7v_y4JcgQx0yDxaujVjya3SHR0IKrTAUROOFF_0hzgERSP-_mA
- [24] E. Strubell, A. Ganesh, and A. McCallum, “Energy and policy considerations for deep learning in nlp,” no. arXiv:1906.02243, Jun. 2019, arXiv:1906.02243 [cs]. [En línea]. Disponible en: <http://arxiv.org/abs/1906.02243>
- [25] E. Union, “Regulation (eu) 2024/1689 of the european parliament and of the council of 27 june 2024 laying down harmonised rules on artificial intelligence (artificial intelligence act) and amending certain union legislative acts,” 2024. [En línea]. Disponible en: <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX%3A32024R1689>
- [26] “Estudio de viabilidad de proyectos.” [En línea]. Disponible en: https://www.esan.edu.pe/conexion-esan/estudio-de-viabilidad-de-proyectos-por-que-es-importante?utm_source=chatgpt.com
- [27] R. Wirth and J. Hipp, “Crisp-dm: Towards a standard process model for data mining.”
- [28] Aug. 2021. [En línea]. Disponible en: <https://www.ibm.com/docs/es/spss-modeler/saas?topic=preparation-data-overview>

- [29] P. Resnick and H. R. Varian, "Recommender systems," *Commun. ACM*, vol. 40, no. 3, p. 56–58, Mar. 1997.
- [30] M. J. Pazzani and D. Billsus, *Content-Based Recommendation Systems*. Berlin, Heidelberg: Springer, 2007, p. 325–341. [En línea]. Disponible en: https://doi.org/10.1007/978-3-540-72079-9_10
- [31] D. Goldberg, D. Nichols, B. M. Oki, and D. Terry, "Using collaborative filtering to weave an information tapestry," *Commun. ACM*, vol. 35, no. 12, p. 61–70, Dec. 1992.
- [32] Y. Koren, R. Bell, and C. Volinsky, "Matrix factorization techniques for recommender systems," *Computer*, vol. 42, no. 8, p. 30–37, Aug. 2009.
- [33] R. Burke, "Hybrid recommender systems: Survey and experiments," *User Modeling and User-Adapted Interaction*, vol. 12, no. 4, p. 331–370, Nov. 2002.
- [34] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, and T.-S. Chua, "Neural collaborative filtering," no. arXiv:1708.05031, Aug. 2017, arXiv:1708.05031 [cs]. [En línea]. Disponible en: <http://arxiv.org/abs/1708.05031>
- [35] B. Perozzi, R. Al-Rfou, and S. Skiena, "Deepwalk: Online learning of social representations," in *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*, Aug. 2014, p. 701–710, arXiv:1403.6652 [cs]. [En línea]. Disponible en: <http://arxiv.org/abs/1403.6652>
- [36] A. Grover and J. Leskovec, "node2vec: Scalable feature learning for networks," no. arXiv:1607.00653, Jul. 2016, arXiv:1607.00653 [cs]. [En línea]. Disponible en: <http://arxiv.org/abs/1607.00653>
- [37] M. A. Chatti, M. Guesmi, and A. Muslim, "Visualization for recommendation explainability: A survey and new perspectives," no. arXiv:2305.11755, Jun. 2024, arXiv:2305.11755 [cs]. [En línea]. Disponible en: <http://arxiv.org/abs/2305.11755>
- [38] M. Vlachos and D. Svonava, "Graph embeddings for movie visualization and recommendation," Dec. 2012. [En línea]. Disponible en: <https://research.ibm.com/publications/graph-embeddings-for-movie-visualization-and-recommendation>
- [39] Y. Jin, K. Seipp, E. Duval, and K. Verbert, "Go with the flow: Effects of transparency and user control on targeted advertising using flow charts," in *Proceedings of the International Working Conference on Advanced Visual Interfaces*, ser. AVI '16. New York, NY, USA: Association for Computing Machinery, Jun. 2016, p. 68–75. [En línea]. Disponible en: <https://doi.org/10.1145/2909132.2909269>

- [40] J. Vig, S. Sen, and J. Riedl, “Tagsplanations: explaining recommendations using tags,” in *Proceedings of the 14th international conference on Intelligent user interfaces*, ser. IUI ’09. New York, NY, USA: Association for Computing Machinery, Feb. 2009, p. 47–56. [En línea]. Disponible en: <https://doi.org/10.1145/1502650.1502661>
- [41] R. He and J. McAuley, “Vbpr: Visual bayesian personalized ranking from implicit feedback,” no. arXiv:1510.01784, Oct. 2015, arXiv:1510.01784 [cs]. [En línea]. Disponible en: <http://arxiv.org/abs/1510.01784>
- [42] C. Szegedy, S. Ioffe, V. Vanhoucke, and A. Alemi, “Inception-v4, inception-resnet and the impact of residual connections on learning,” no. arXiv:1602.07261, Aug. 2016, arXiv:1602.07261 [cs]. [En línea]. Disponible en: <http://arxiv.org/abs/1602.07261>
- [43] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, “Imagenet: A large-scale hierarchical image database,” in *2009 IEEE Conference on Computer Vision and Pattern Recognition*, Jun. 2009, p. 248–255. [En línea]. Disponible en: <https://ieeexplore.ieee.org/document/5206848>
- [44] S. Rendle, C. Freudenthaler, Z. Gantner, and L. Schmidt-Thieme, “Bpr: Bayesian personalized ranking from implicit feedback,” no. arXiv:1205.2618, May 2012, arXiv:1205.2618 [cs]. [En línea]. Disponible en: <http://arxiv.org/abs/1205.2618>
- [45] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, “Going deeper with convolutions,” no. arXiv:1409.4842, Sep. 2014, arXiv:1409.4842 [cs]. [En línea]. Disponible en: <http://arxiv.org/abs/1409.4842>
- [46] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” no. arXiv:1512.03385, Dec. 2015, arXiv:1512.03385 [cs]. [En línea]. Disponible en: <http://arxiv.org/abs/1512.03385>
- [47] J. Bekker and J. Davis, “Learning from positive and unlabeled data: a survey,” *Machine Learning*, vol. 109, no. 4, p. 719–760, Apr. 2020, arXiv:1811.04820 [cs].
- [48] C. Shorten and T. M. Khoshgoftaar, “A survey on image data augmentation for deep learning,” *Journal of Big Data*, vol. 6, no. 1, p. 60, Jul. 2019.
- [49] R. Gozalo-Brizuela and E. C. Garrido-Merchán, “A survey of generative ai applications,” no. arXiv:2306.02781, Jun. 2023, arXiv:2306.02781 [cs]. [En línea]. Disponible en: <http://arxiv.org/abs/2306.02781>
- [50] S. S. Mullick, S. Datta, and S. Das, “Generative adversarial minority oversampling,” in *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*. Seoul,

- Korea (South): IEEE, Oct. 2019, p. 1695–1704. [En línea]. Disponible en: <https://ieeexplore.ieee.org/document/9008836/>
- [51] J. Rocchio, “Xxiii. relevance feedback in information retrieval.”
- [52] X. Su and T. M. Khoshgoftaar, “A survey of collaborative filtering techniques,” *Advances in Artificial Intelligence*, vol. 2009, no. 1, p. 421425, 2009.
- [53] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, “High-resolution image synthesis with latent diffusion models,” no. arXiv:2112.10752, Apr. 2022, arXiv:2112.10752 [cs]. [En línea]. Disponible en: <http://arxiv.org/abs/2112.10752>
- [54] S. Patil, W. Berman, R. Rombach, and P. v. Platen, “amused: An open muse reproduction,” no. arXiv:2401.01808, Jan. 2024, arXiv:2401.01808 [cs]. [En línea]. Disponible en: <http://arxiv.org/abs/2401.01808>
- [55] H. Chang, H. Zhang, J. Barber, A. J. Maschinot, J. Lezama, L. Jiang, M.-H. Yang, K. Murphy, W. T. Freeman, M. Rubinstein, Y. Li, and D. Krishnan, “Muse: Text-to-image generation via masked generative transformers,” no. arXiv:2301.00704, Jan. 2023, arXiv:2301.00704 [cs]. [En línea]. Disponible en: <http://arxiv.org/abs/2301.00704>
- [56] V. Hondru, F. A. Croitoru, S. Minaee, R. T. Ionescu, and N. Sebe, “Masked image modeling: A survey,” no. arXiv:2408.06687, Jan. 2025, arXiv:2408.06687 [cs]. [En línea]. Disponible en: <http://arxiv.org/abs/2408.06687>
- [57] P. W. Code, “Image classification on imagenet.” [En línea]. Disponible en: <https://paperswithcode.com/sota/image-classification-on-imagenet>
- [58] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, “An image is worth 16x16 words: Transformers for image recognition at scale,” no. arXiv:2010.11929, Jun. 2021, arXiv:2010.11929 [cs]. [En línea]. Disponible en: <http://arxiv.org/abs/2010.11929>
- [59] G. Shani and A. Gunawardana, *Evaluating Recommendation Systems*. Boston, MA: Springer US, 2011, p. 257–297. [En línea]. Disponible en: https://doi.org/10.1007/978-0-387-85820-3_8
- [60] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, “Bleu: a method for automatic evaluation of machine translation,” in *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, P. Isabelle, E. Charniak, and D. Lin, Eds. Philadelphia, Pennsylvania, USA: Association for Computational Linguistics, Jul. 2002, p. 311–318. [En línea]. Disponible en: <https://aclanthology.org/P02-1040/>

- [61] M. Popović, “chrF: character n-gram f-score for automatic mt evaluation,” in *Proceedings of the Tenth Workshop on Statistical Machine Translation*, O. Bojar, R. Chatterjee, C. Federmann, B. Haddow, C. Hokamp, M. Huck, V. Logacheva, and P. Pecina, Eds. Lisbon, Portugal: Association for Computational Linguistics, Sep. 2015, p. 392–395. [En línea]. Disponible en: <https://aclanthology.org/W15-3049/>
- [62] O. Vinyals, A. Toshev, S. Bengio, and D. Erhan, “Show and tell: A neural image caption generator,” 2015, p. 3156–3164. [En línea]. Disponible en: https://www.cv-foundation.org/openaccess/content_cvpr_2015/html/Vinyals_Show_and_Tell_2015_CVPR_paper.html
- [63] B. Thompson, “Sentence similarity and machine translation,” Ph.D. dissertation, Johns Hopkins University, 2020.
- [64] A. Gretton, K. M. Borgwardt, M. J. Rasch, B. Schölkopf, and A. Smola, “A kernel two-sample test,” *Journal of Machine Learning Research*, vol. 13, no. 25, p. 723–773, 2012.
- [65] T. t. p. g. i. S. a. n. l. f. t. i. g. b. c. o. c. D. t. v. u. cycles and S. C. D. M. u.-t.-D. D. T. R. i. t. Text, “Topic: Tripadvisor.” [En línea]. Disponible en: <https://www.statista.com/topics/3443/tripadvisor/>
- [66] “Pep 8 – style guide for python code | peps.python.org.” [En línea]. Disponible en: <https://peps.python.org/pep-0008/>